Local Search and Optimization



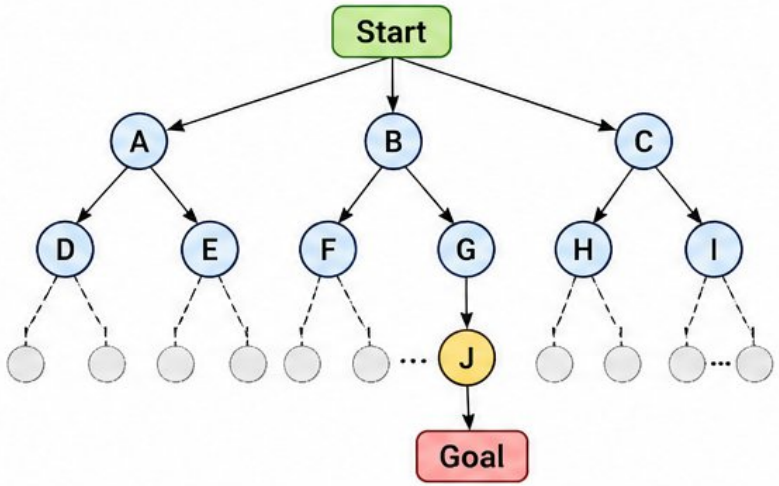
Tree Search vs. Local Search

*In many problems, we don't care how we reached the solution — **only the solution itself matters**.*

Example: 8–Queens Problem

TREE SEARCH

Explores many states and keeps track of the **path, frontier** and **parent nodes**.



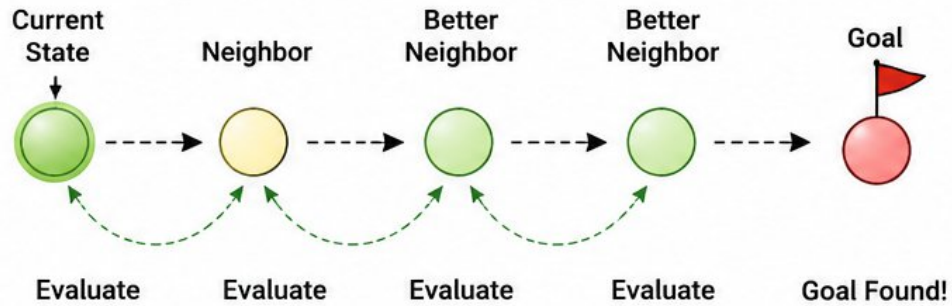
Stores:

- Full Path (from Start to Goal)
- Frontier (unexplored states)
- Parent / Back Pointers

VS

LOCAL SEARCH

Works with a **single current state** and moves to one of its **neighboring states**.



Stores Only:

- Current State

In optimization problems, how we reached the solution is usually irrelevant.

We only care about the final solution.

Constraint Satisfaction Problem (CSP)

- **A CSP consists of**
  - Variables
    - Western Australia (WA)
    - Norther Territory (NT)
    - Queensland (Q)
    - South Australia (SA)
    - New South Wales (NSW)
    - Victoria (V)
    - Tasmania (T)
  - Domain
    - Red
    - Green
    - Blue
  - Constraint
    - Adjacent regions can not have the same color

Nodes: Variables
Edges: Constraint

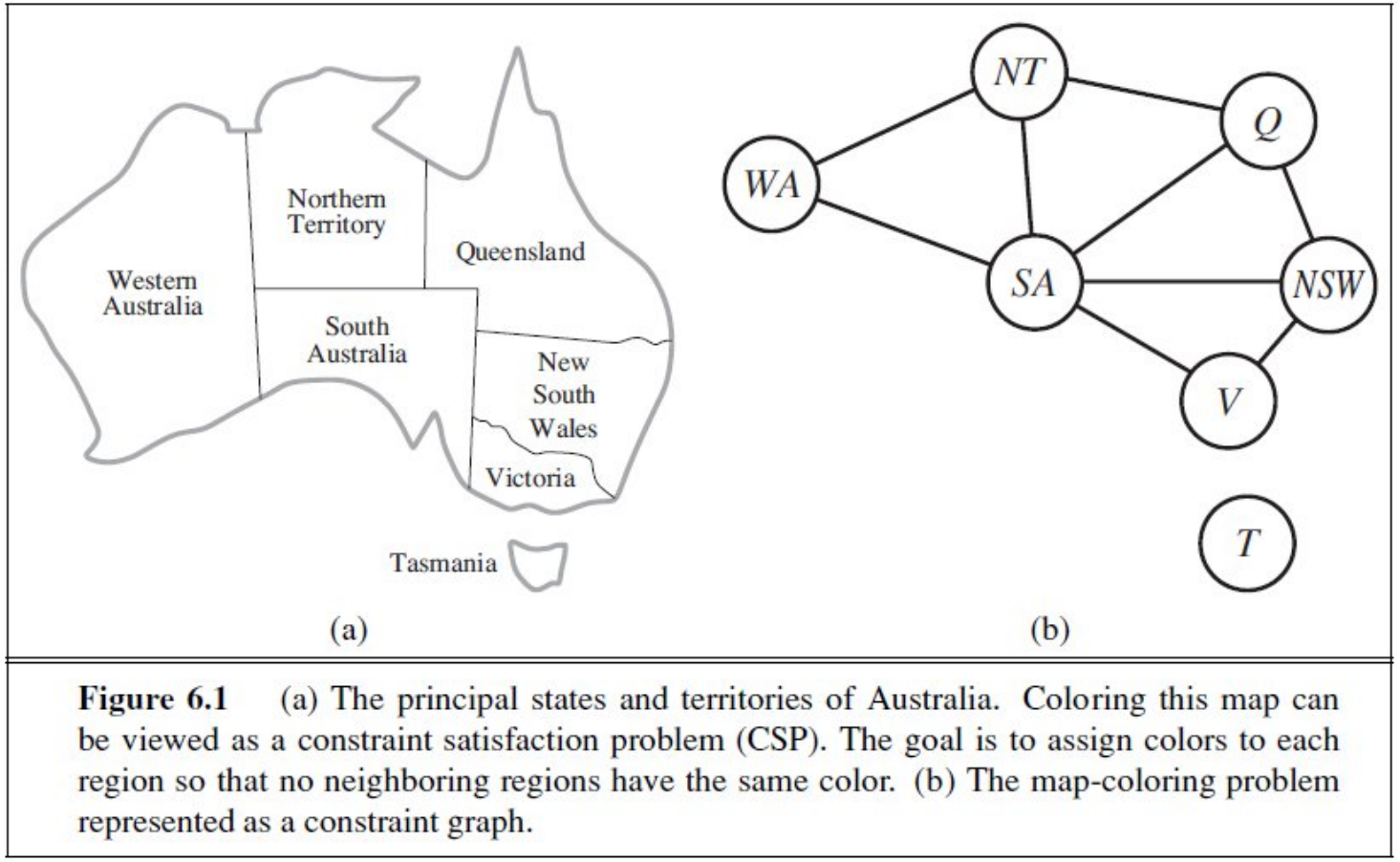
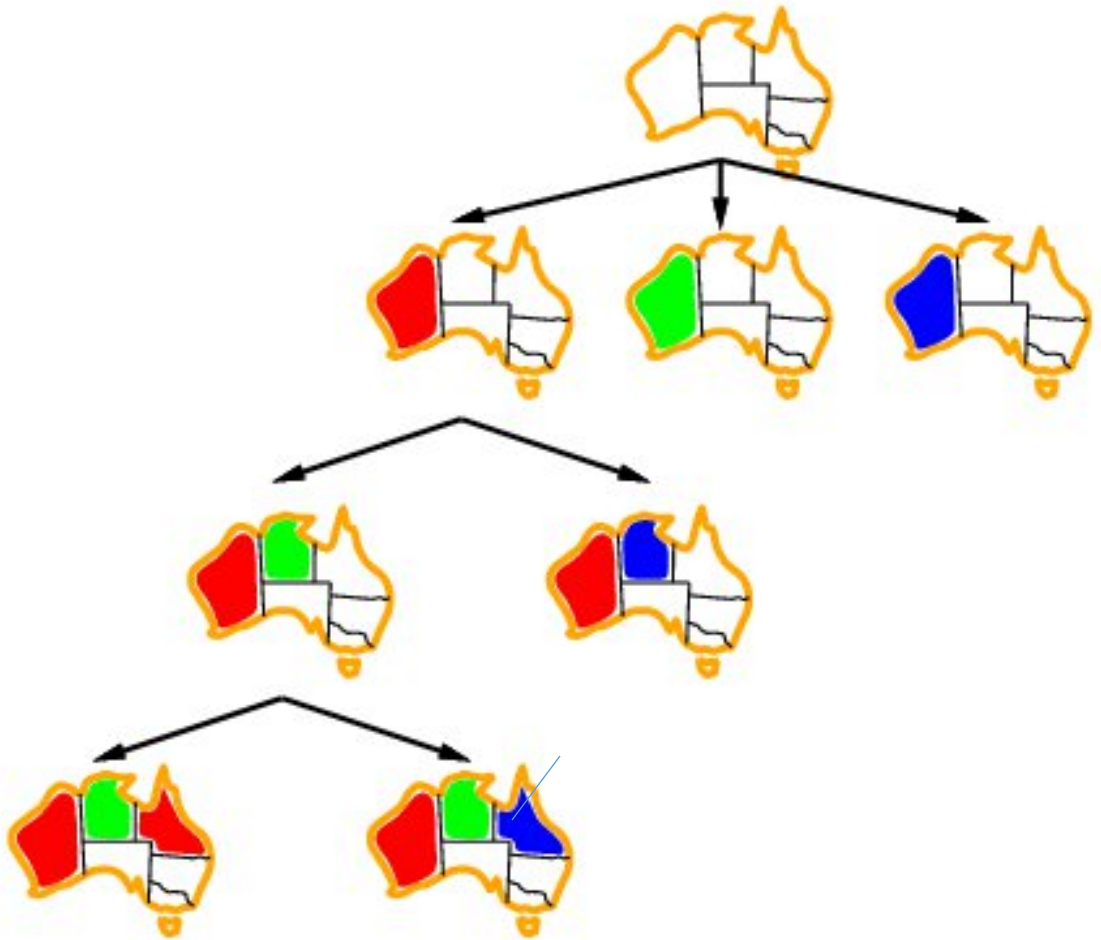
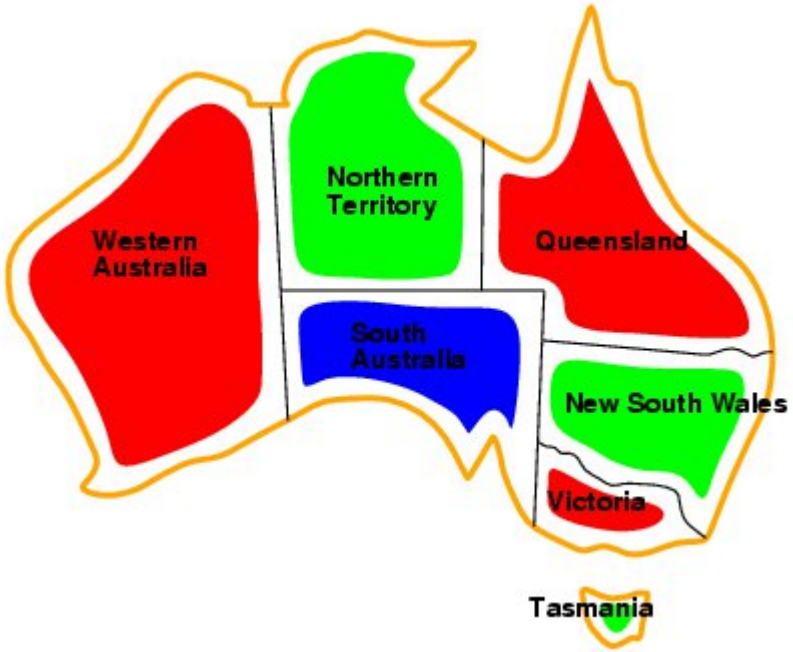


Figure 6.1 (a) The principal states and territories of Australia. Coloring this map can be viewed as a constraint satisfaction problem (CSP). The goal is to assign colors to each region so that no neighboring regions have the same color. (b) The map-coloring problem represented as a constraint graph.

Example



Example: Map-Coloring



- Solutions are complete and consistent assignments, e.g., WA = red, NT = green,Q = red,NSW = green,V = red,SA = blue,T = green

Local Search Algorithms

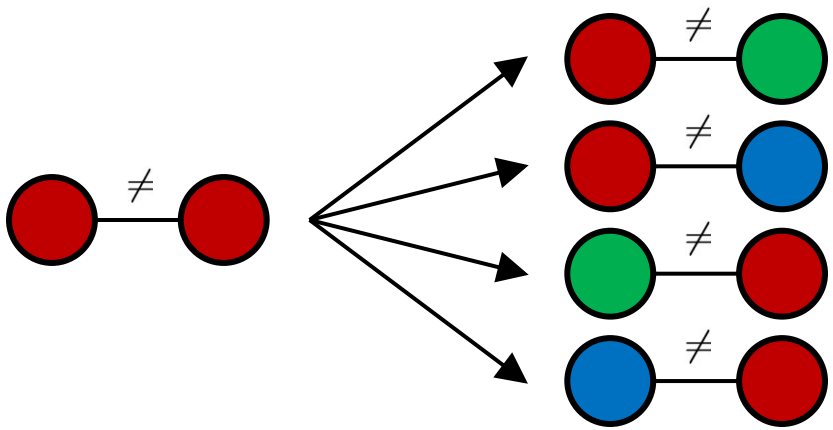
- When there is no need to memorize the path to the goal, just to find the goal.
- Cost of the path and finding optimal path to the goal is not required.
- Start from a single current state and check the neighboring states
- Consumes little memory
- Useful with large state space.

Tree Search Vs. Local Search

Tree Search	**Local Search**
Keeps full path	Only current state
Uses memory	Very low memory
Complete	Not always

Local Search

- Tree search keeps unexplored alternatives on the fringe (ensures completeness)
- Local search: improve a single option until you can't make it better (no fringe!)
- New successor function: local changes



- Generally much faster and more memory efficient (but incomplete and suboptimal)

Local Search Methods – Applications

- Applicable when seeking Goal State & don't care how to get there.
  E.g.,
  - N-queens,
  - finding shortest/cheapest round trips
    - **(Travel Salesman Problem, Vehicle Routing Problem)**
  - finding models of propositional formulae (SAT solvers)
  - VLSI layout, planning, scheduling, time-tabling, . . .
  - map coloring,
  - resource allocation

When should we use local search?

- Traditional Search
  - Path to the goal is important
  - Finds the complete route
  - Stores explored nodes
  - Uses more memory

  - Examples
    - GPS Navigation
    - Maze Solving
    - Robot Navigation

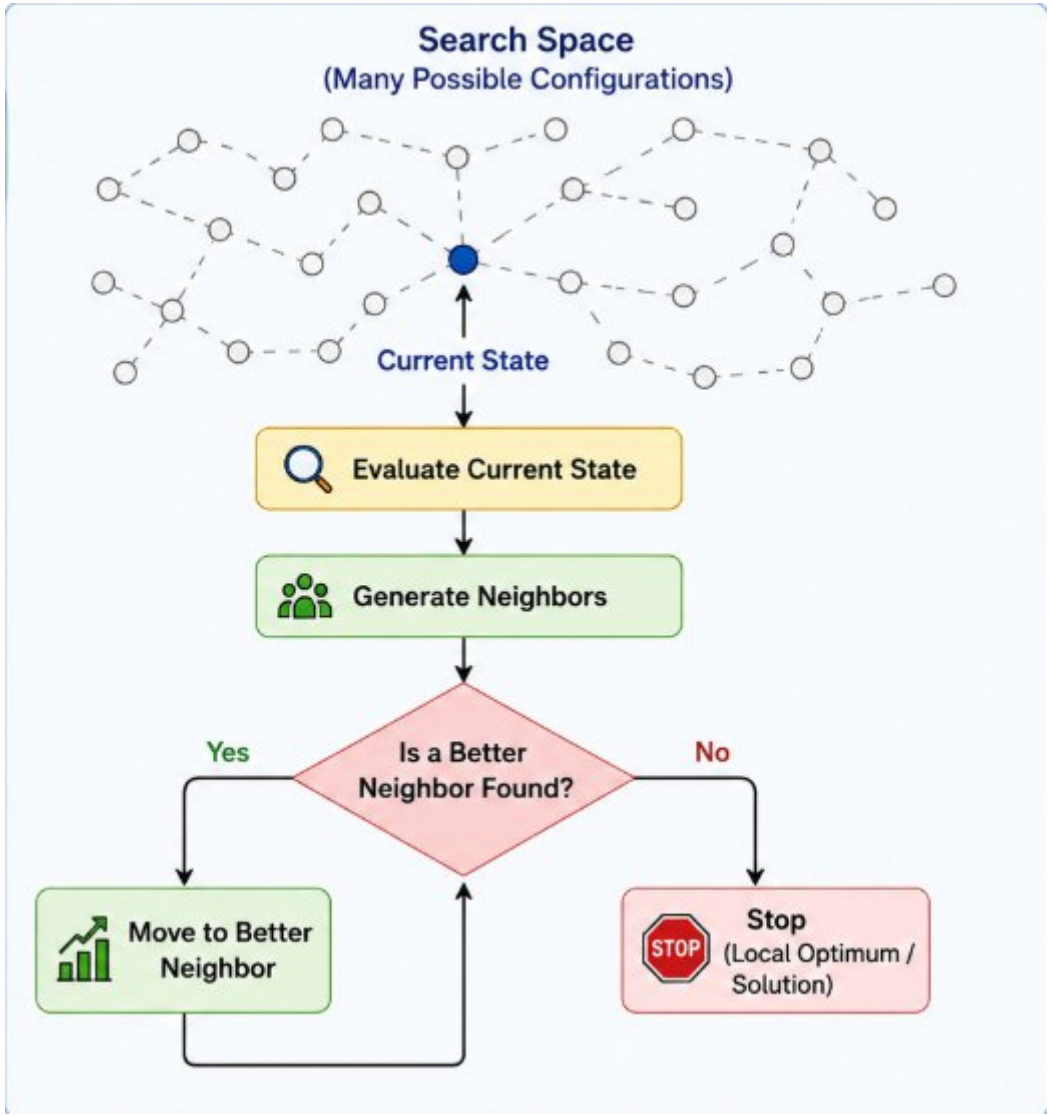
- Local Search
  - Only the final state matters
  - No need to remember the path
  - Keeps only the current state
  - Very little memory

  - Examples
    - N-queens
    - Map coloring

Local Search Algorithm

- Local search algorithms work as follows:
  1. Pick a “solution” from the search space and evaluate it. Define this as the current solution.
  2. Apply a transformation to the current solution to generate and evaluate a new solution.
  3. If the new solution is better than the current solution then exchange it with the current solution; otherwise discard the new solution.
  4. Repeat steps 2 and 3 until no transformation in the given set improves the current solution

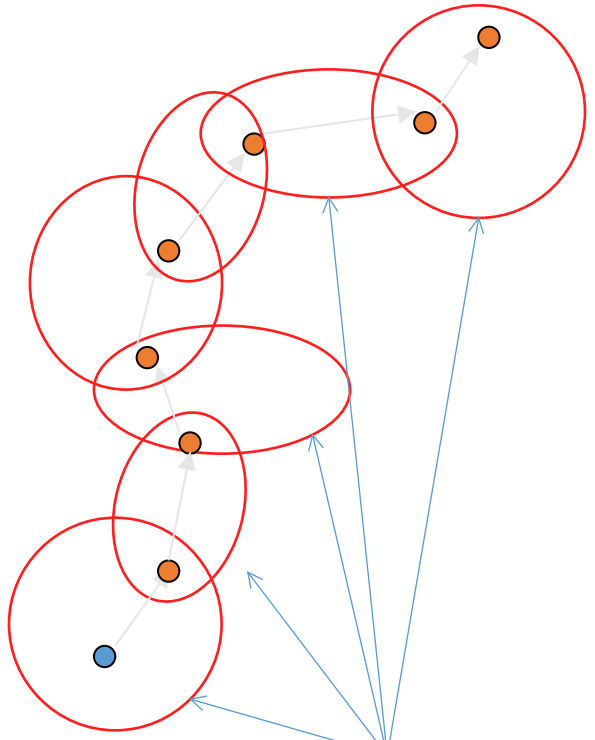
- Note: No back tracking. Why?



Local search

- **Key idea (surprisingly simple):**

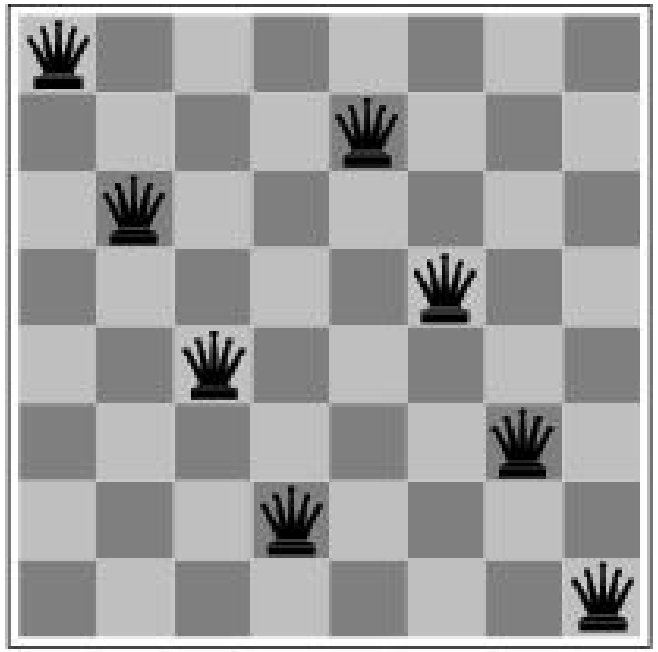
1. Select (random) initial state (generate an initial guess)
2. Make **local modification** to **improve** current state
   - Evaluate current state and move to other states
3. Repeat Step 2 until goal state found (or out of time)



Neighborhoods

Local Search for optimization Problems

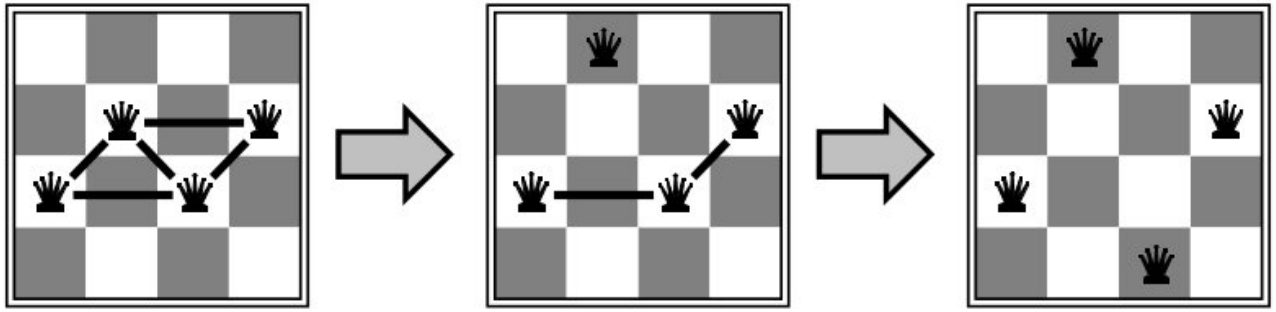
- Optimization Problem
  - Goal is to find the best solution according to an objective function
- Systematic Search
  - Often impractical for huge search spaces
- Iterative Improvement
  - Start with any valid (or invalid) solution and improve it
- Objective Function
  - Measure solution quality and optimize it.



Minimize the number of attacks

Example: n-Queen

- Put N Queens on an $n \times n$ board with no two queens on the same row, column, or diagonal
- Move a queen to reduce number of conflicts

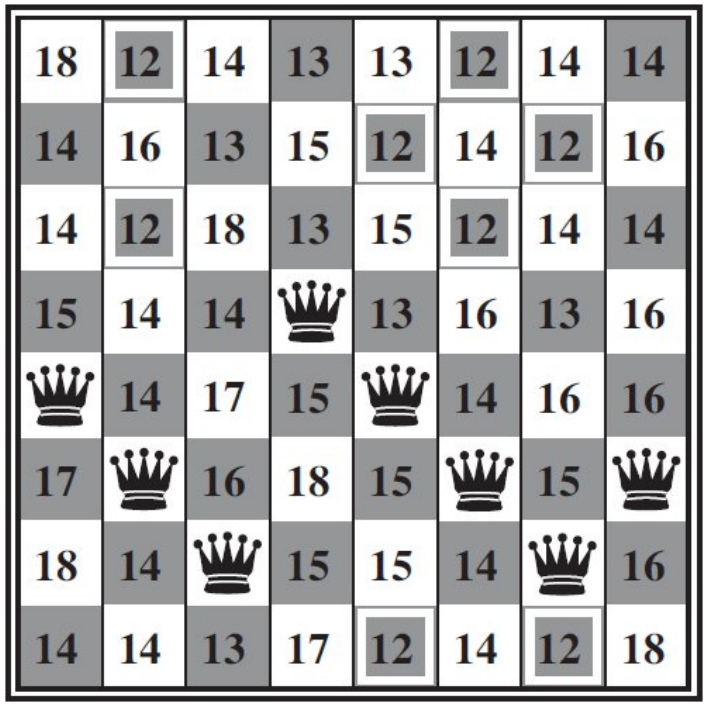


Initial state … Improve it … using local transformations

Almost always solves n-queens problems instantaneously for very large n, e.g., n = 1 million

Example: n-Queen

- Put N Queens on an $n \times n$ board with no two queens on the same row, column, or diagonal

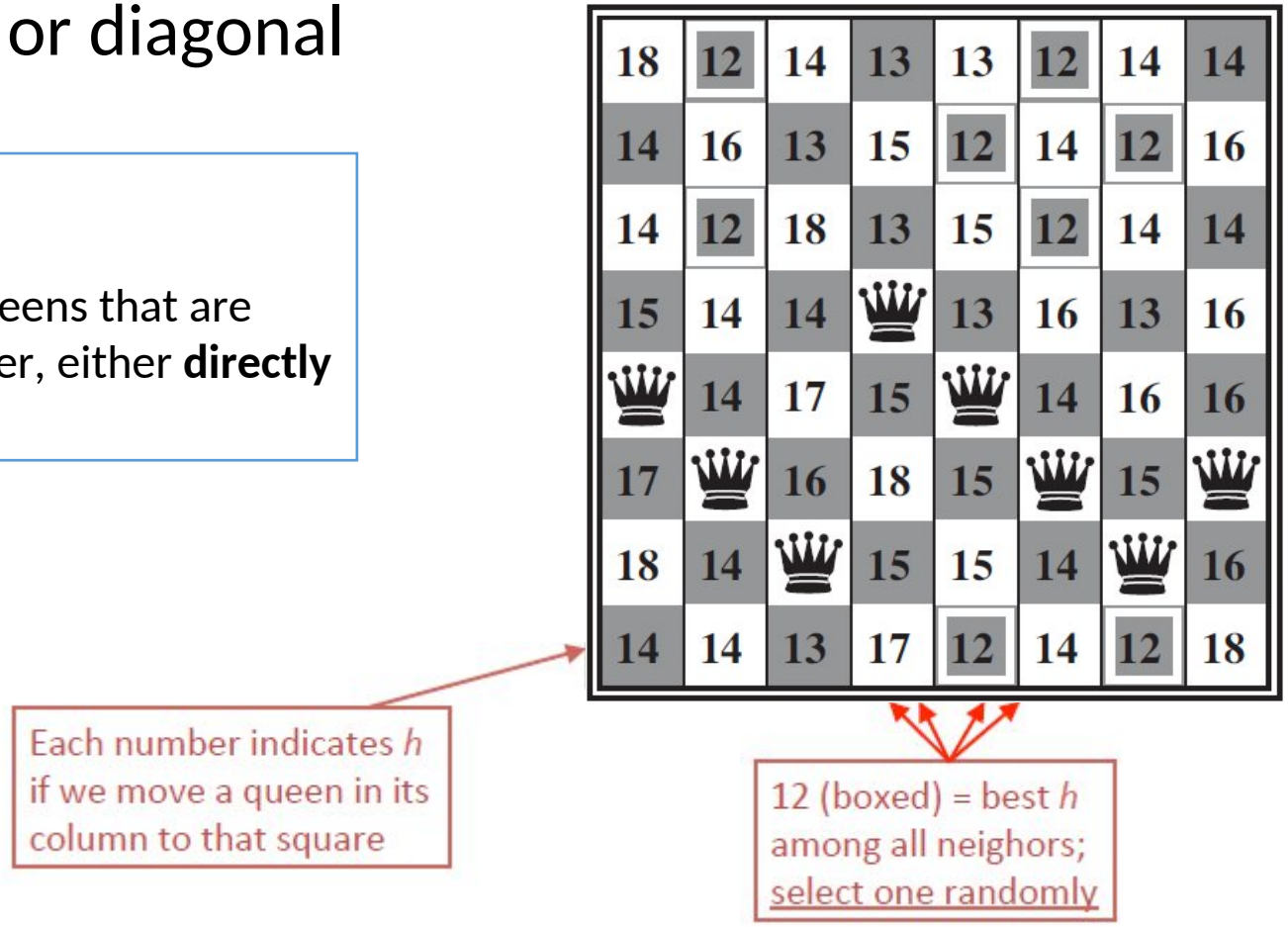


Example: n-Queen

- Put N Queens on an $n \times n$ board with no two queens on the same row, column, or diagonal

HEURISTIC:

h = # of pairs of queens that are attacking each other, either **directly or indirectly**



Example: Travelling Salesman Problem (TSP)

TSP is an optimization problem.

Cities

A salesman has five cities to visit.

Visit each city **exactly once.**

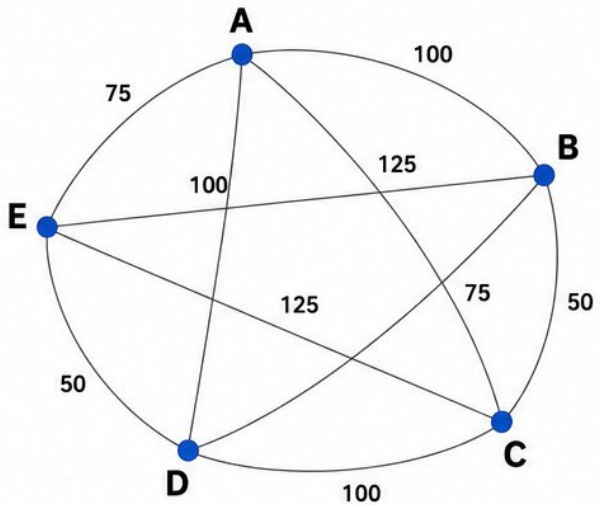
Return Home

After visiting all cities, the salesman must return to the starting city.

Objective

Find the tour (cycle) with the **minimum total travel distance**.

Five Cities and Distances

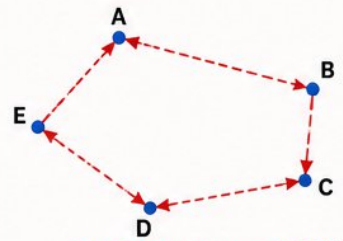


A, B, C, D, E = Cities

Numbers on edges = Distance (e.g., in km)

Initial Tour (Random)

A → C → E → B → D → A

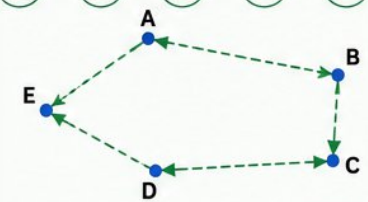


Total Distance = **520 km**

Local Search

Improved Tour (After Local Search)

A → B → C → D → E → A



Total Distance = **365 km**

Better (Shorter Distance)

Objective Function

For TSP:

f(state) = Total Distance of Tour

Goal:

Minimize f(state)

How Local Search Improves the Solution

520 km (Initial) → 440 km → 390 km → 365 km (Better) → Best Tour (Goal)

Distance **decreases** at each step until no further improvement is possible.

Key Takeaway

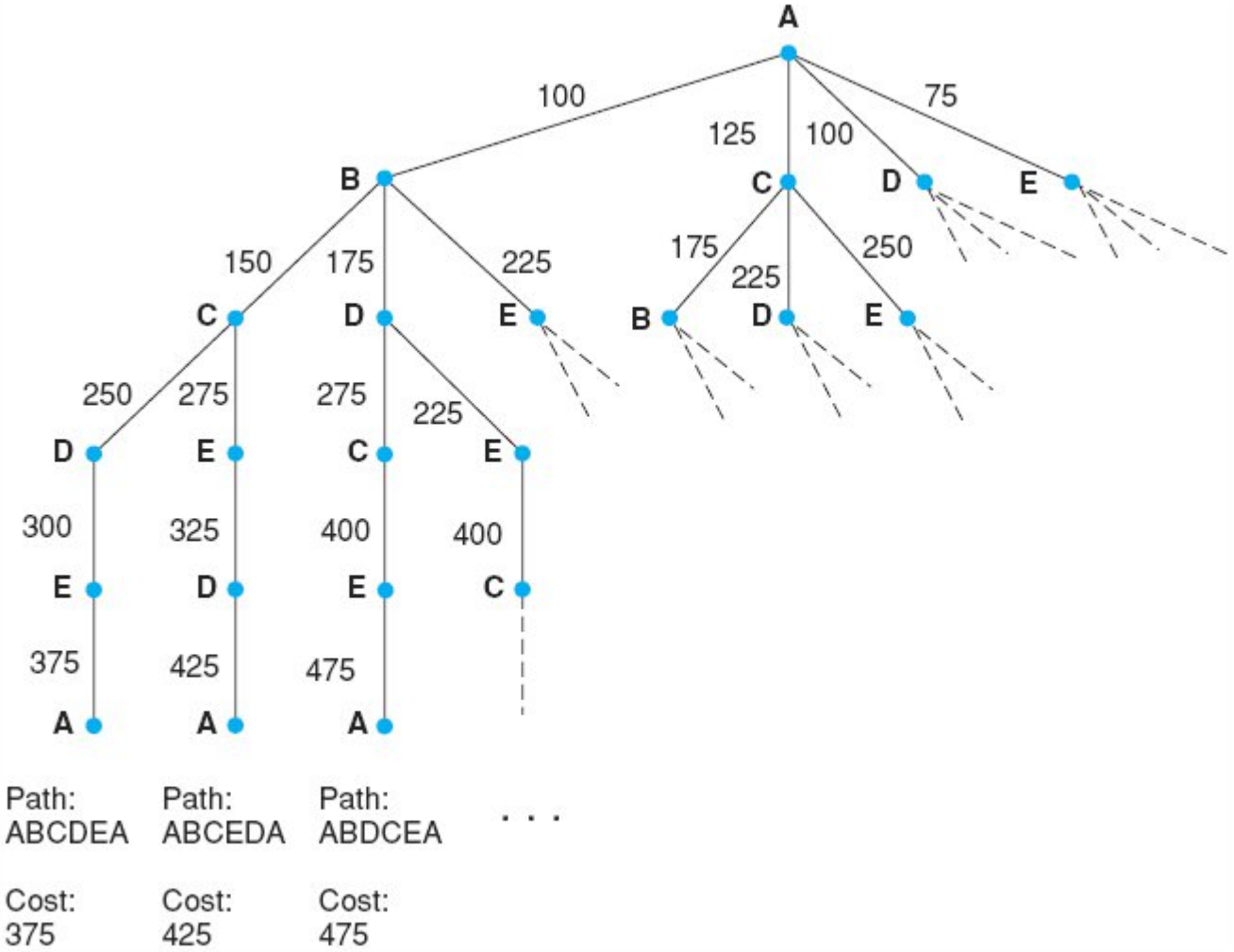
Local search starts with any tour and keeps improving it by reducing the total distance (objective function).

Example: Travelling Salesman Problem



- A Solution: Exhaustive Search
  - (Generate and Test) !!
- The number of all tours is about (n-1)!/2
- If n = 36 the number is about:
- 566573983193072464833325668761600000000
- Not Viable Approach !!

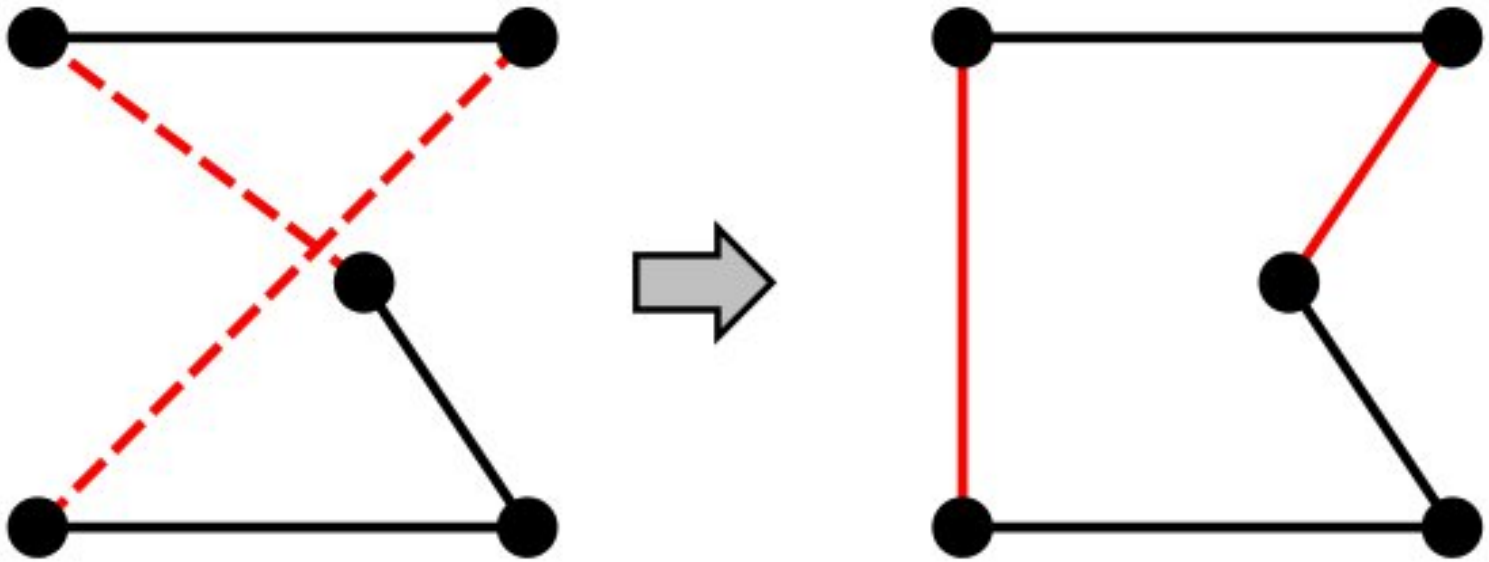
Search for the travelling salesperson problem. Each arc is marked with the total weight of all paths from the start node (A) to its endpoint.



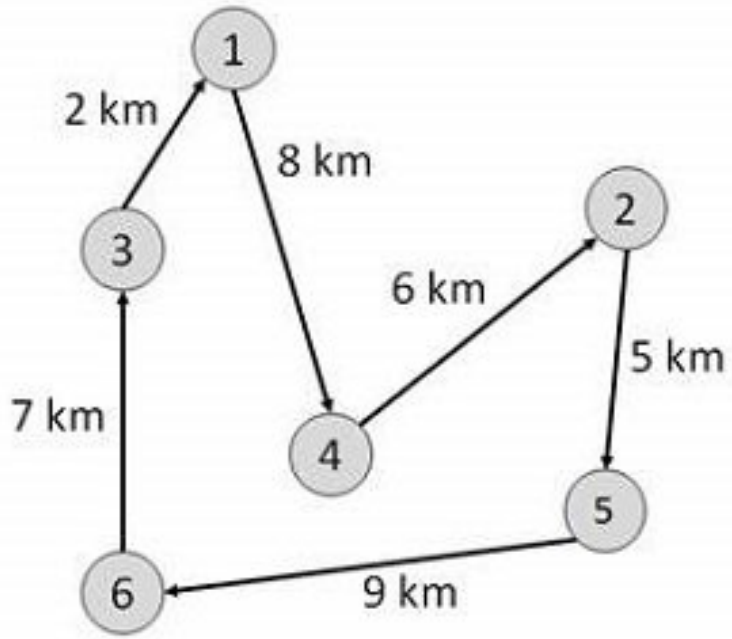
Luger: Artificial Intelligence, 6th edition. © Pearson Education Limited, 2009

Example: Travelling Salesman Problem

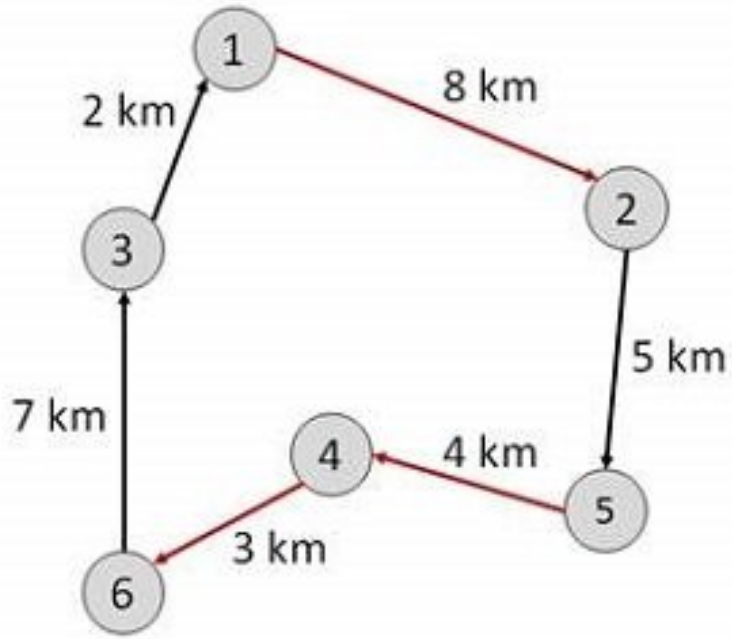
Start with any complete tour, perform pairwise exchanges



Variants of this approach get within 1% of optimal very quickly with thousands of cities



Total Distance = 37km



Total Distance = 31km

Hill Climbing Algorithm

Gradient Ascent/Descent Algorithm



Hill Climbing: The Basic Idea

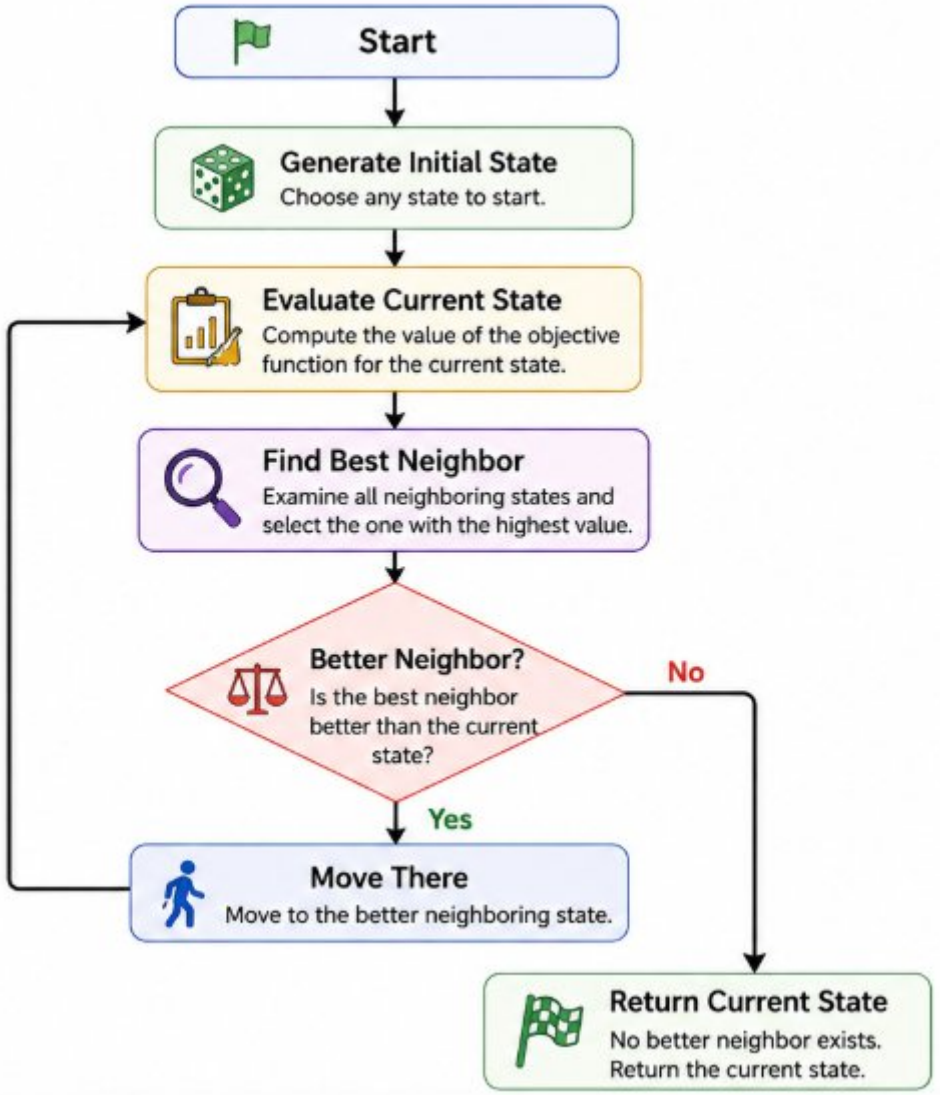
1. Start from any state
2. Move to the best available neighboring state
3. Stop when no better neighbor exists



Hill Climbing always chooses a better neighboring state.

Only local information is used—Hill Climbing cannot see the entire landscape.

Hill Climbing: Algorithm



Definition:

Hill Climbing is a greedy local search algorithm that repeatedly moves to the **best neighboring state** until **no better neighbor exists**.

Key Points:

- Greedy strategy
- Uses local information only
- Stops at a local optimum

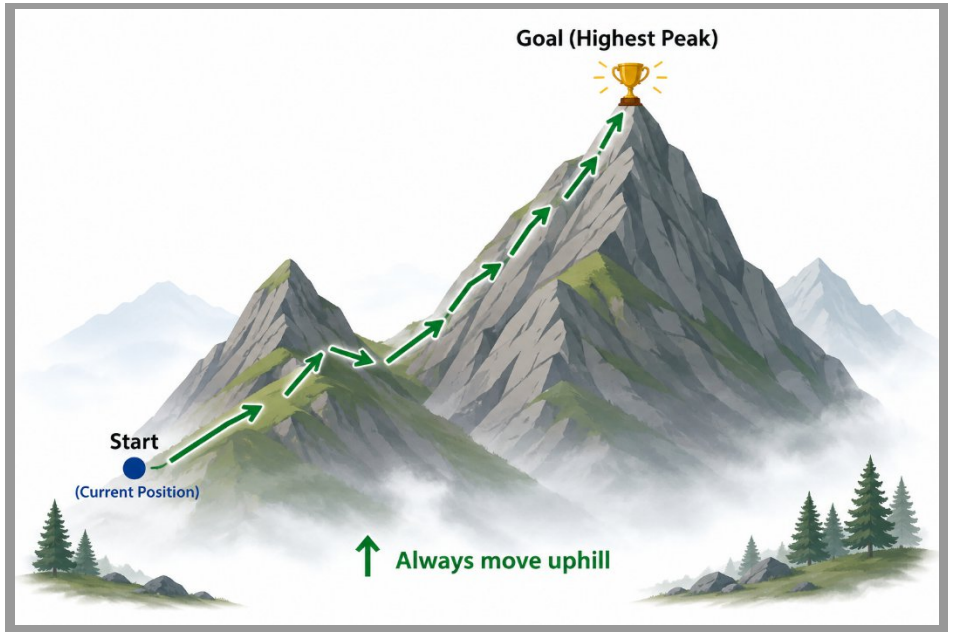
Advantages:

- Simple
- Memory Efficient
- Fast

Hill Climbing

Will hill climbing always reach its highest peak?

- Complete?
- Optimal?



Hill-Climbing Algorithm

1. Pick a random point in the search space
2. Consider all the neighbors of the current state
3. Choose the neighbor with the best quality and move to that state
4. Repeat 2 to 4 until all the neighboring states are of lower quality
5. Return the current state as the solution state.

Hill Climbing Search

- Steepest Ascent Search, Greedy Local Search
- Iteratively maximize value of current state by replacing its successors state that has highest value , as long as possible.

```
function HILL-CLIMBING(problem) returns a state that is a local maximum

  current ← MAKE-NODE(problem.INITIAL-STATE)
  loop do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE ≤ current.VALUE then return current.STATE
    current ← neighbor
```

Figure 4.2 The hill-climbing search algorithm, which is the most basic local search technique. At each step the current node is replaced by the best neighbor; in this version, that means the neighbor with the highest VALUE, but if a heuristic cost estimate h is used, we would find the neighbor with the lowest h.

Hill Climbing Search

Tree Search	Hill Climbing
Expands many nodes	Expands only the current state
Stores parent and sibling nodes	Stores only the current state
Can backtrack	No backtracking
Maintains a search tree	No Search Tree
Higher memory usage	Very low memory usage

Hill Climbing is memory efficient because it never stores the entire search tree.

Hill Climbing on 8-puzzle

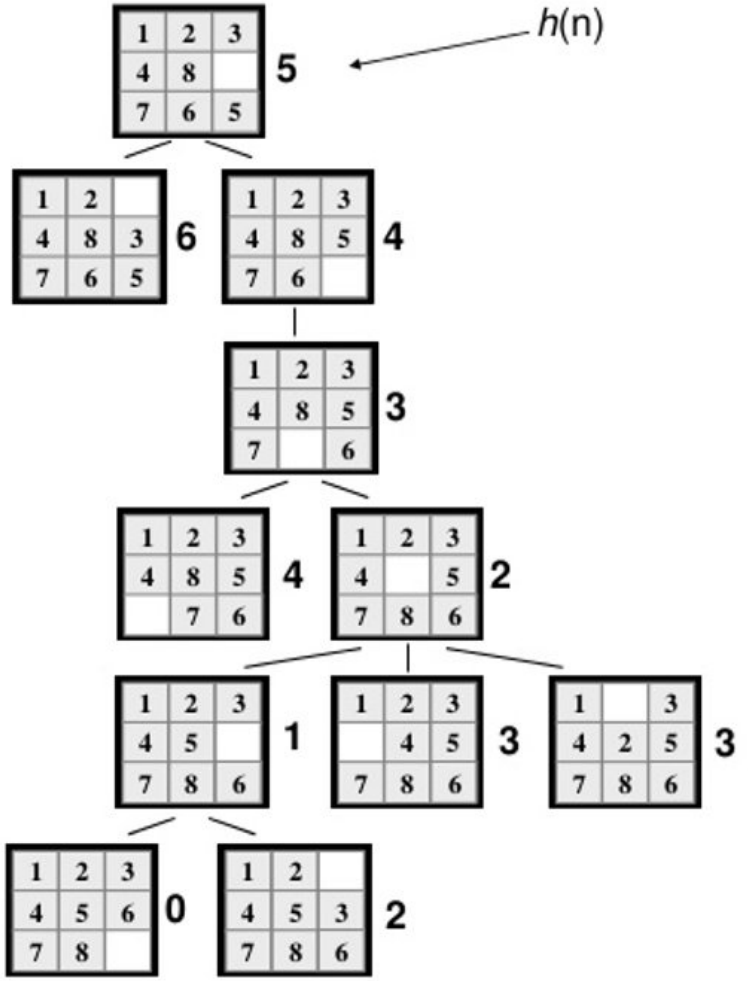
- Use heuristic to evaluate each state
- Generate neighboring states
- Move to a neighbor with better heuristic value
- Heuristic: total moves needed for each tile to reach goal state

1	2	3
4	8	
7	6	5

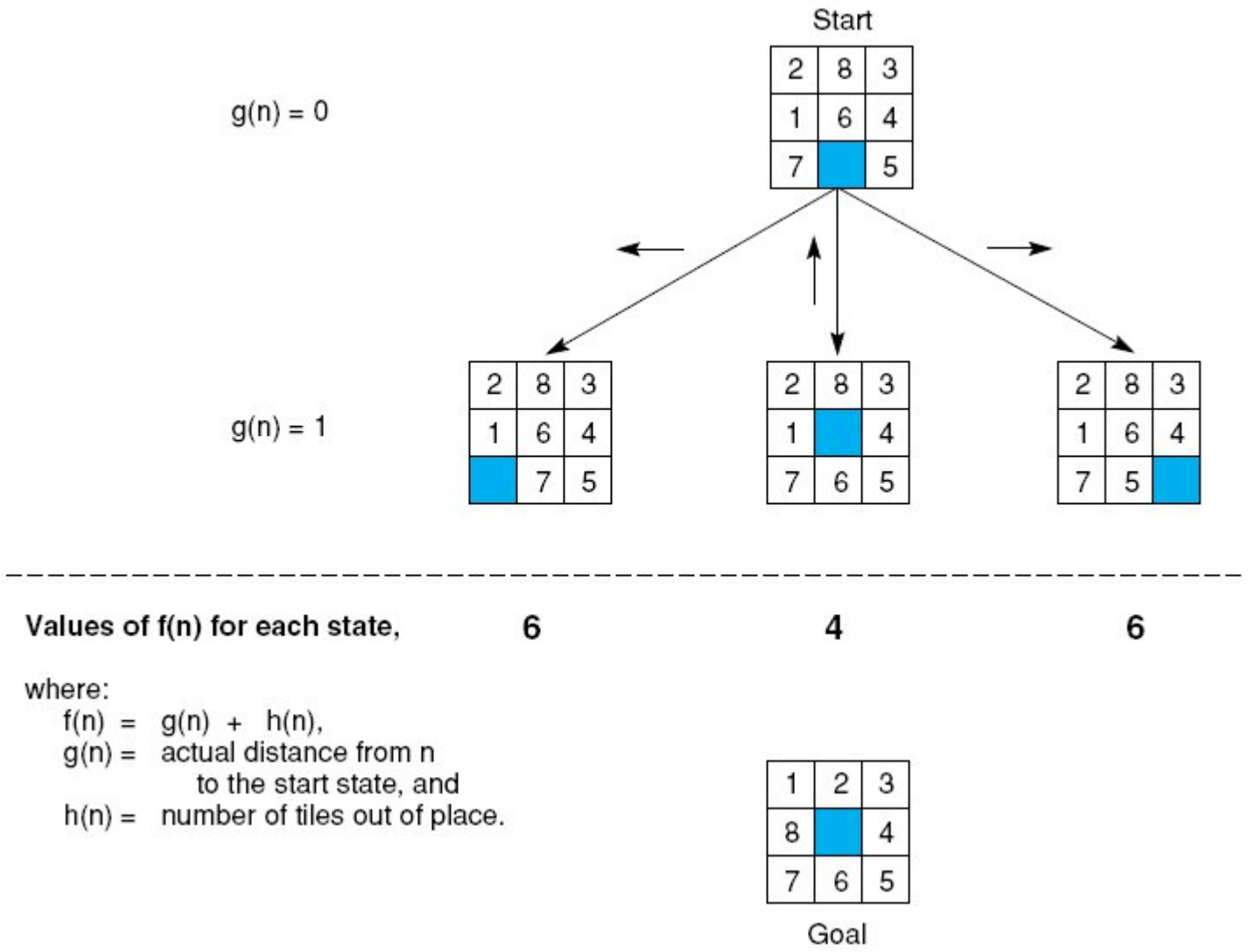
Current State

1	2	3
4	5	6
7	8	

Goal State

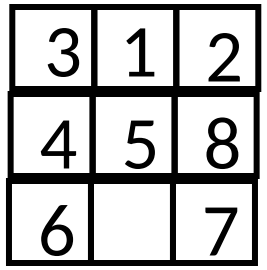


The heuristic f applied to states in the 8-puzzle.



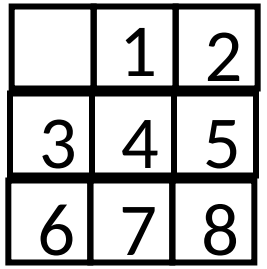
Hill climbing example (*minimizing* h)

start



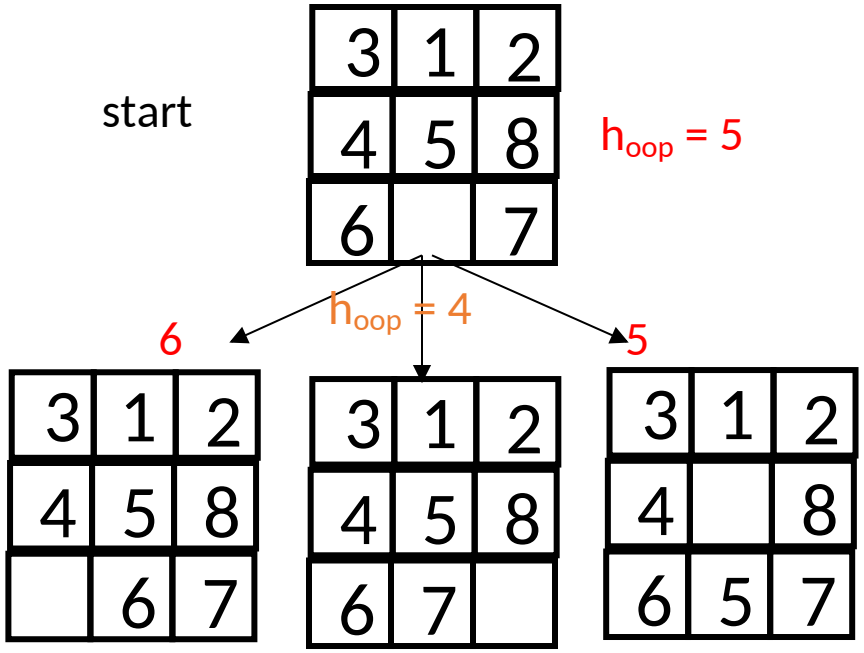
$h_{oop} = 5$

goal

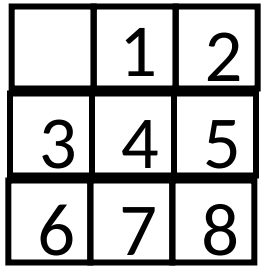


Hill climbing example (*minimizing* h)

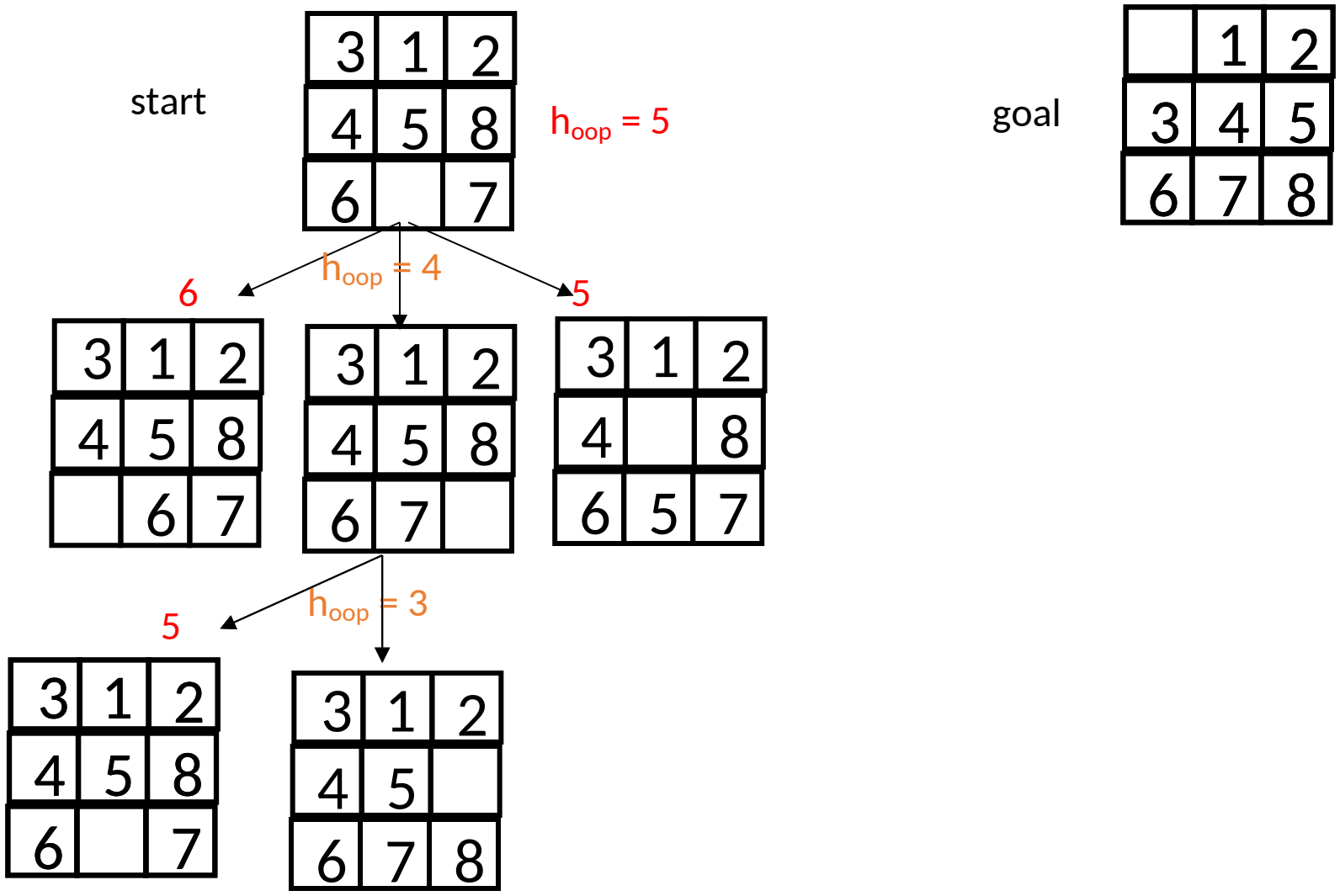
start



goal

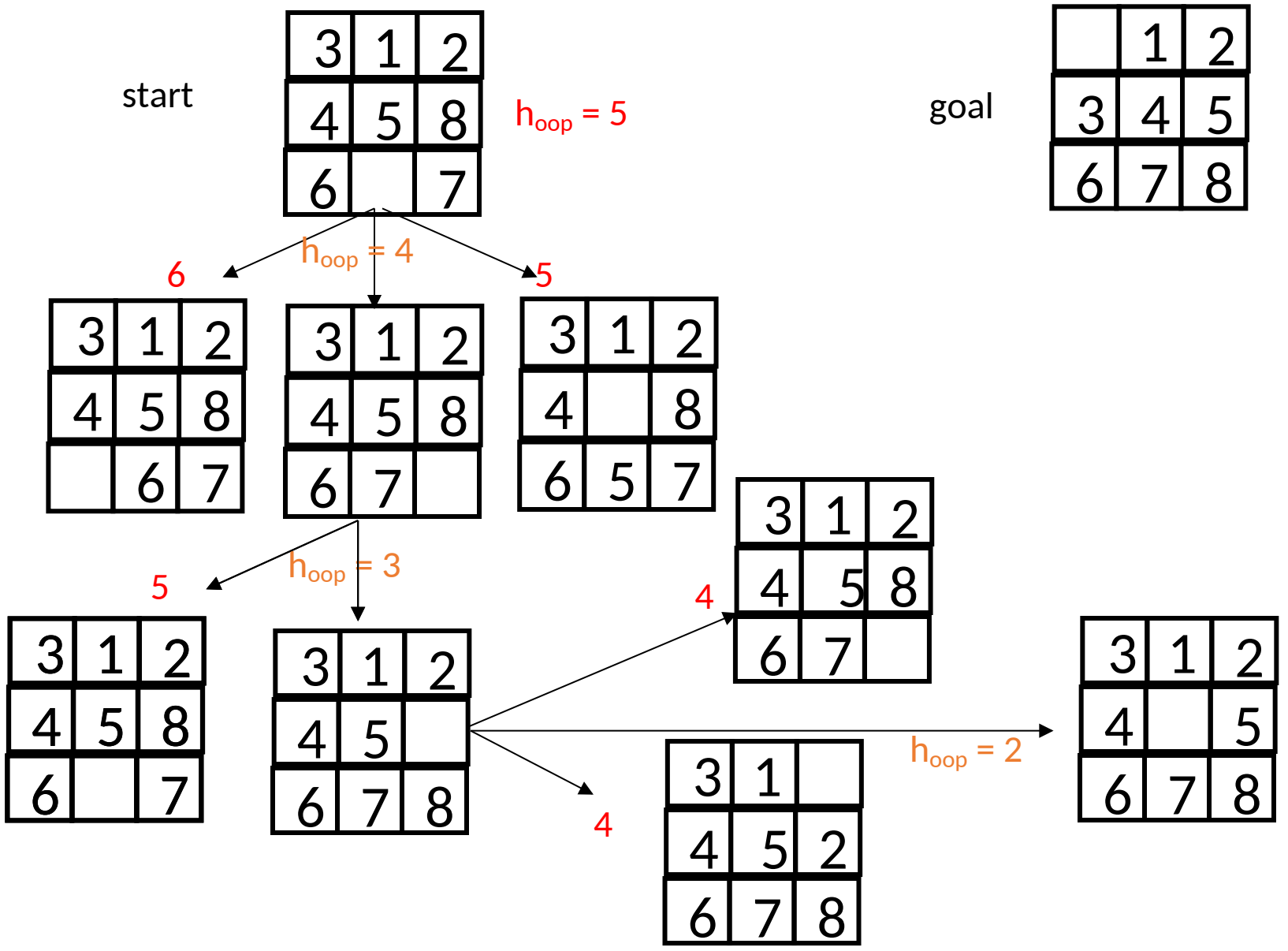


Hill climbing example (*minimizing* h)

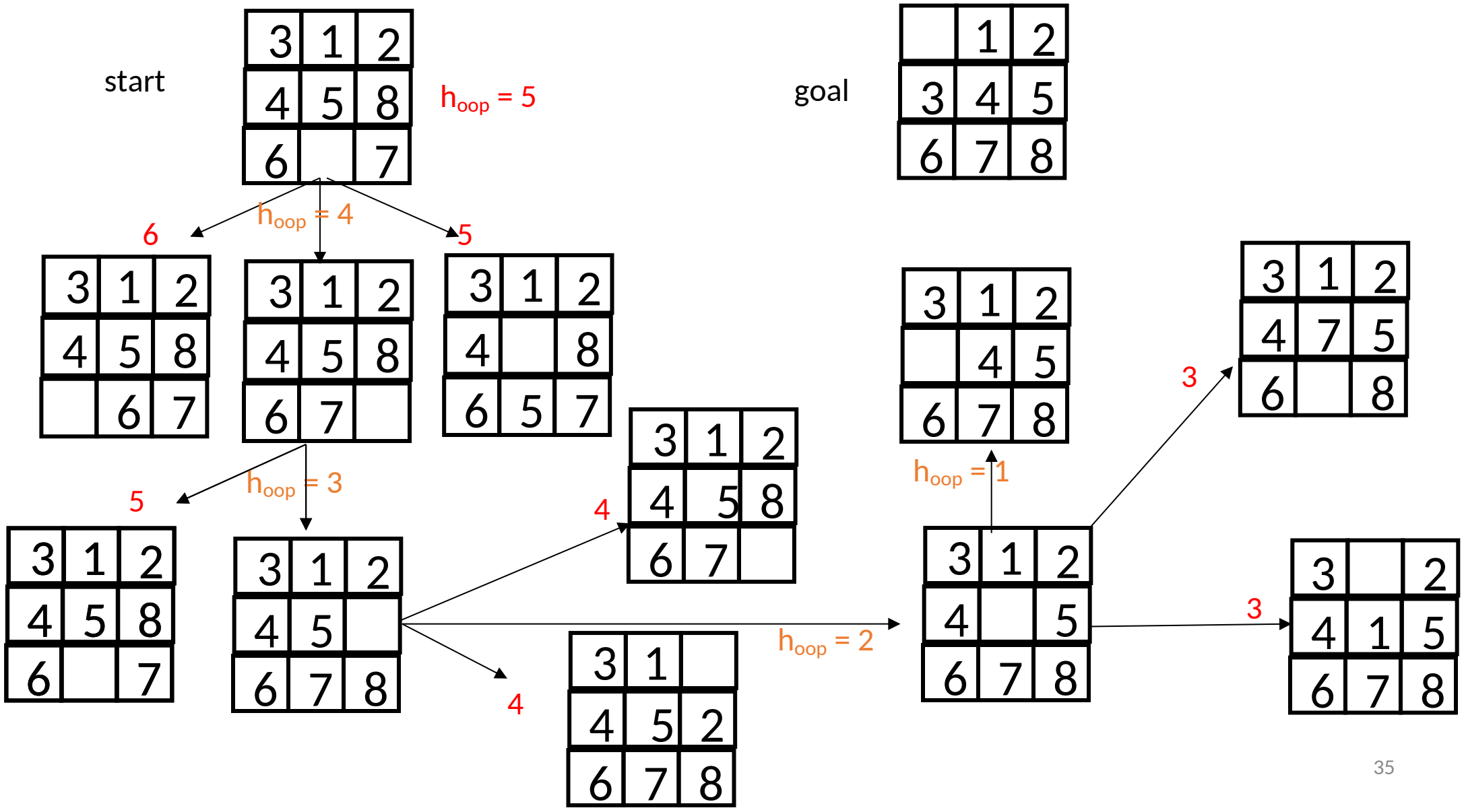


goal

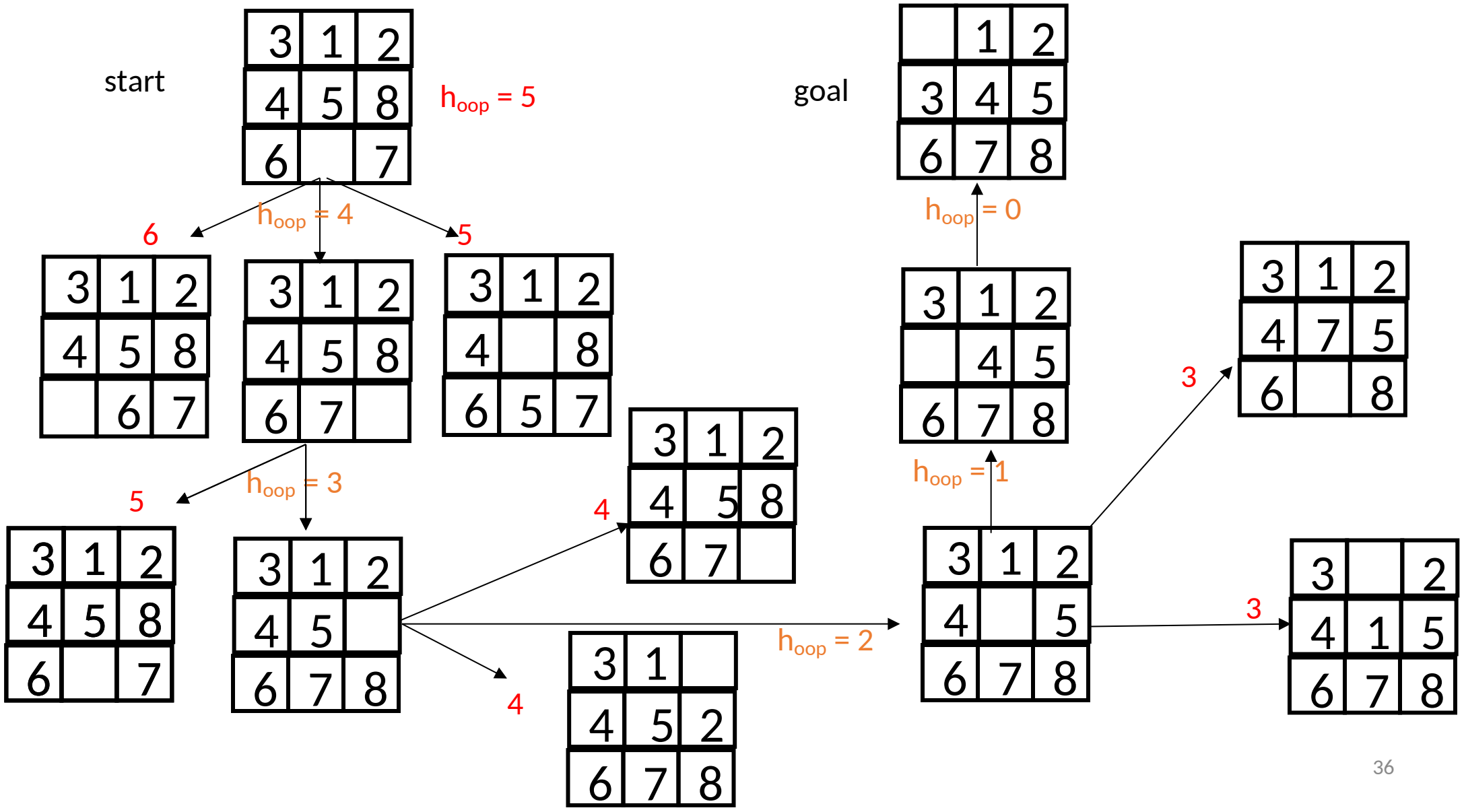
Hill climbing example (*minimizing* h)

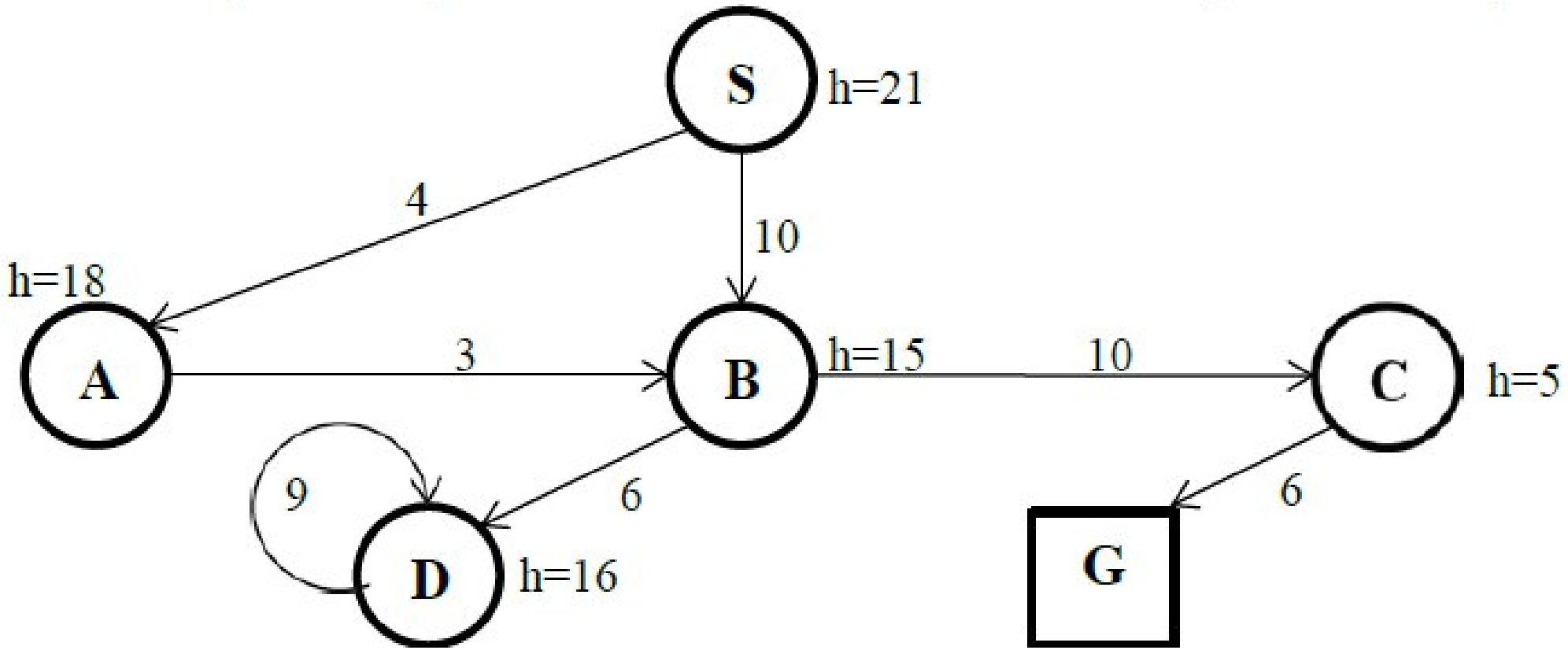


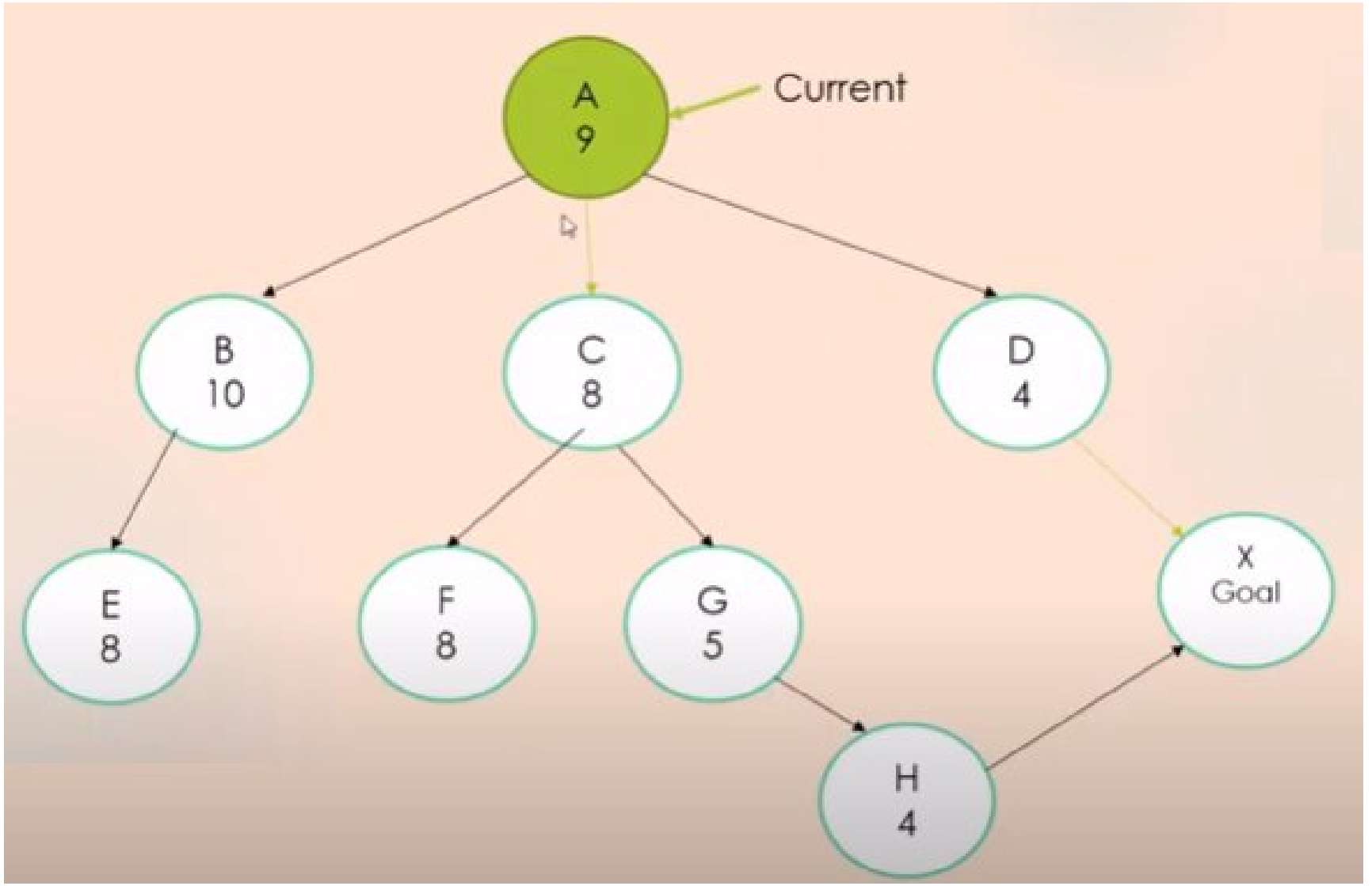
Hill climbing example (*minimizing* h)



Hill climbing example (*minimizing* h)

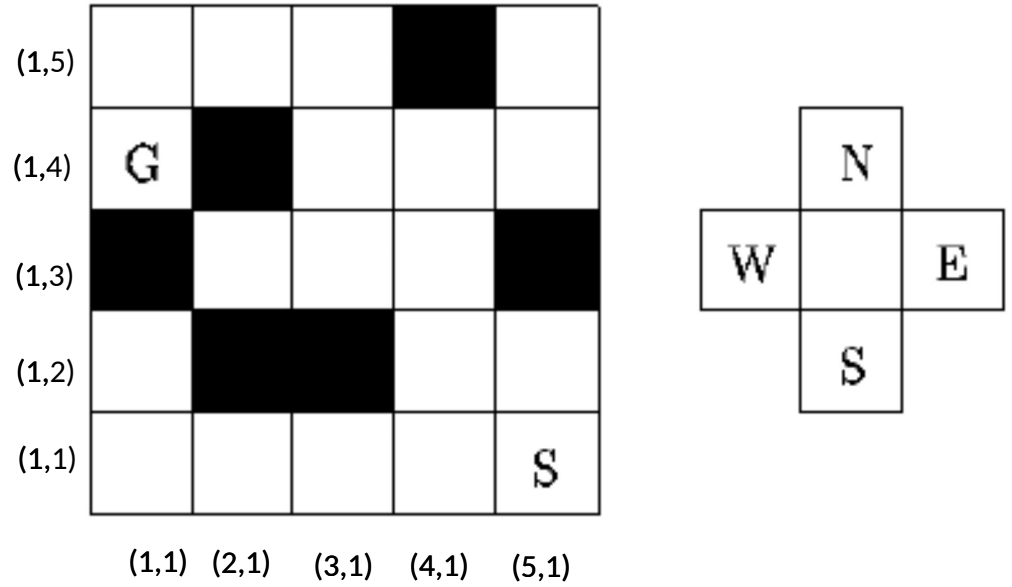


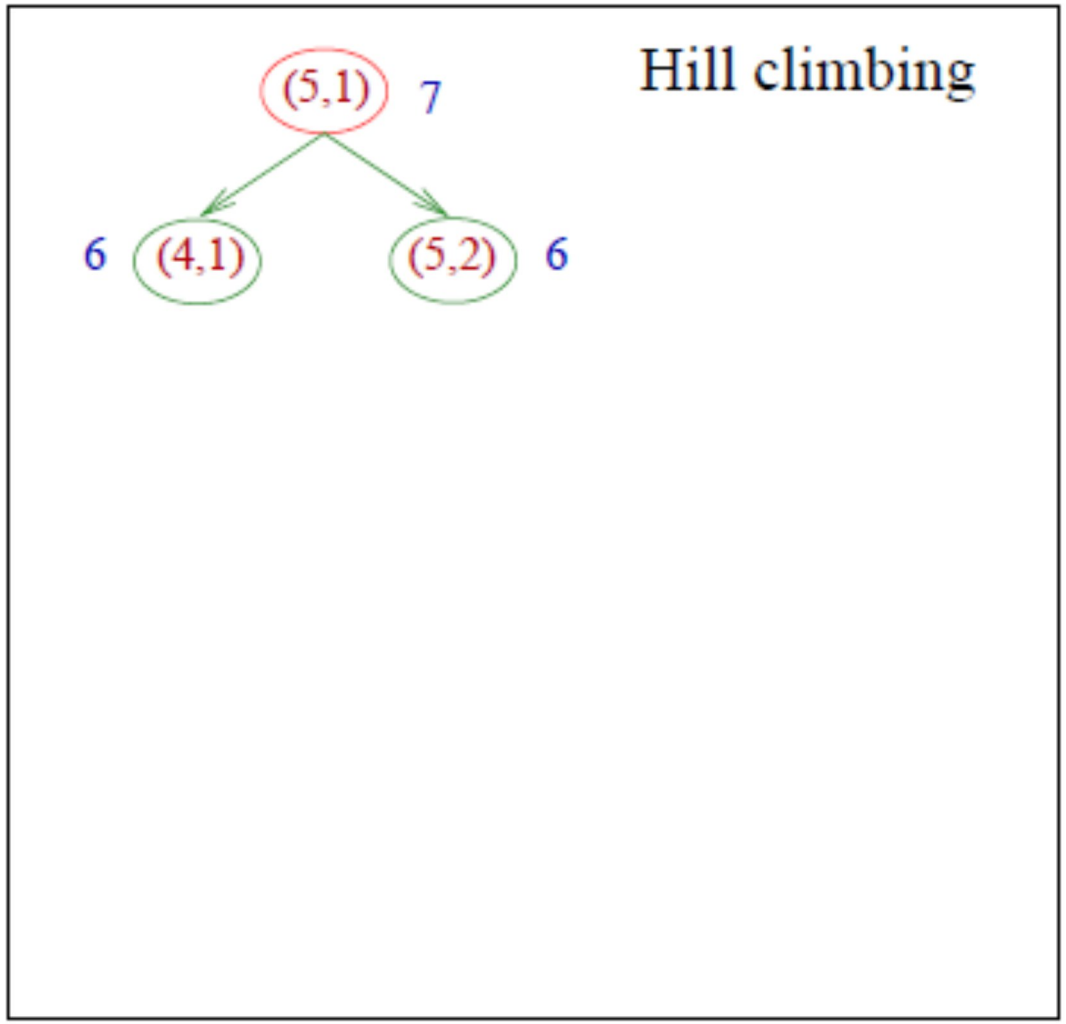

S
h=21
4
10
h=18
A
3
B
h=15
10
C
h=5
9
6
6
D
h=16
G


A
9
Current
B
10
C
8
D
4
E
8
F
8
G
5
X
Goal
H
4

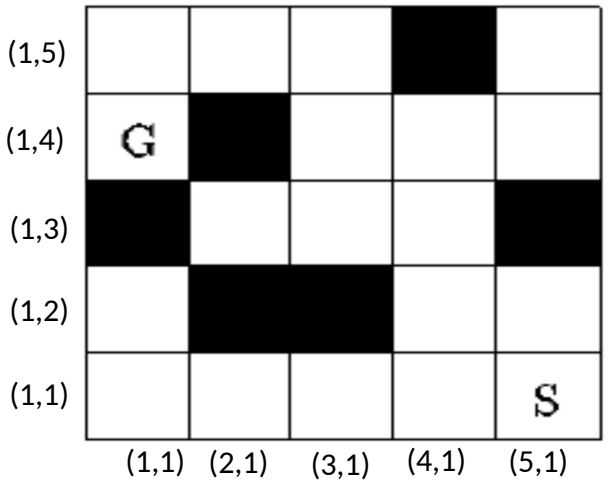
Example of hill-climbing for the maze problem

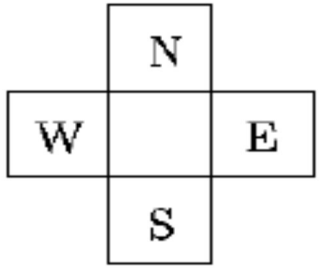
- Consider the following maze:
- The problem is to get from the start node S to the goal node G, by moving horizontally and vertically and avoiding the black obstacles in the above maze.

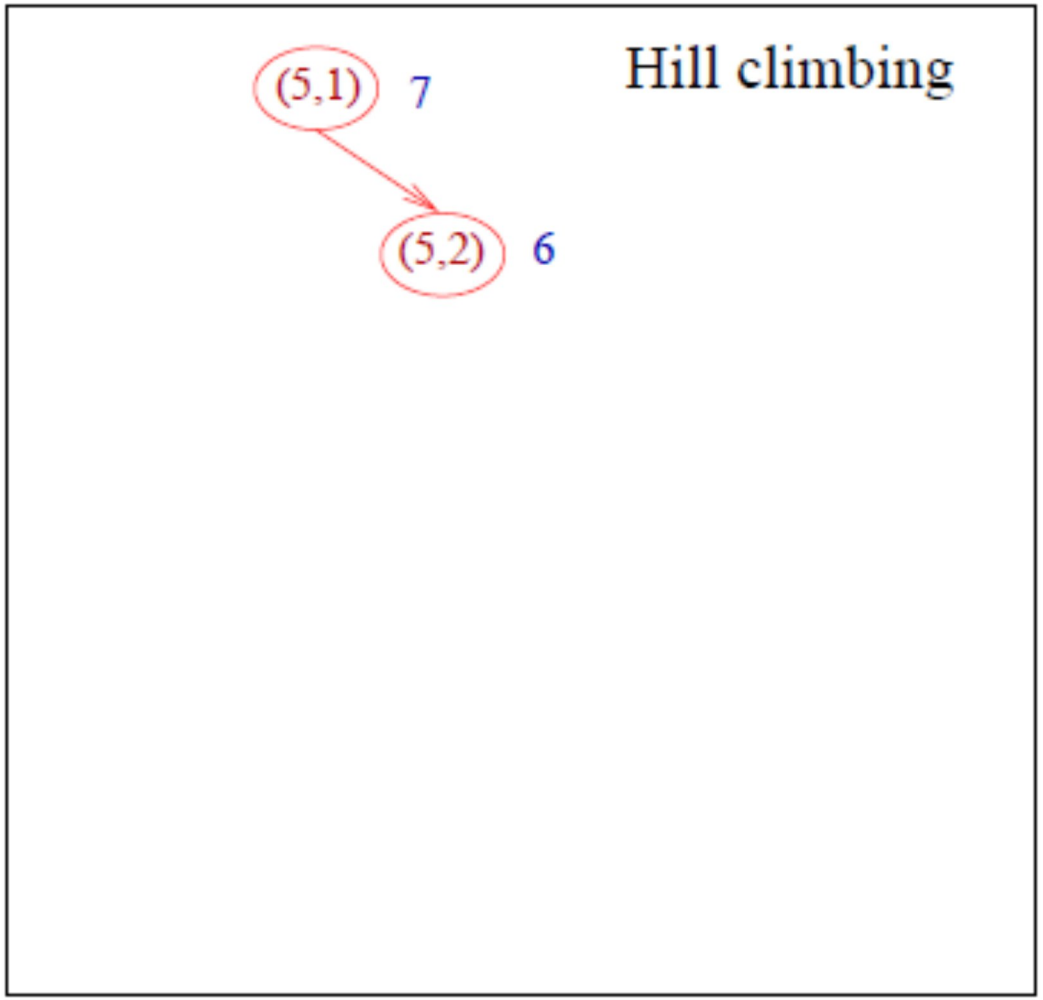




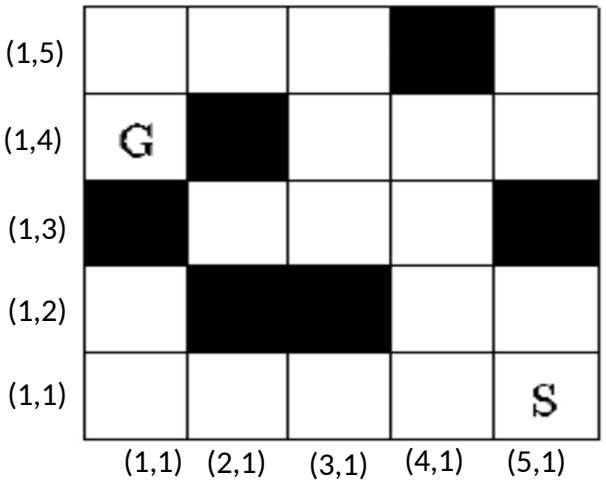
- In the figures describing the steps of the algorithm
- each position will be denoted by a pair of numbers which are the horizontal and vertical coordinates (that is, for (x, y), x denotes the horizontal coordinate and y denotes the vertical coordinate).
- The Manhattan distance between the points (x, y) and (z, t) is given by $d((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2|$.

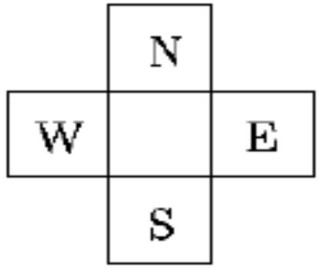


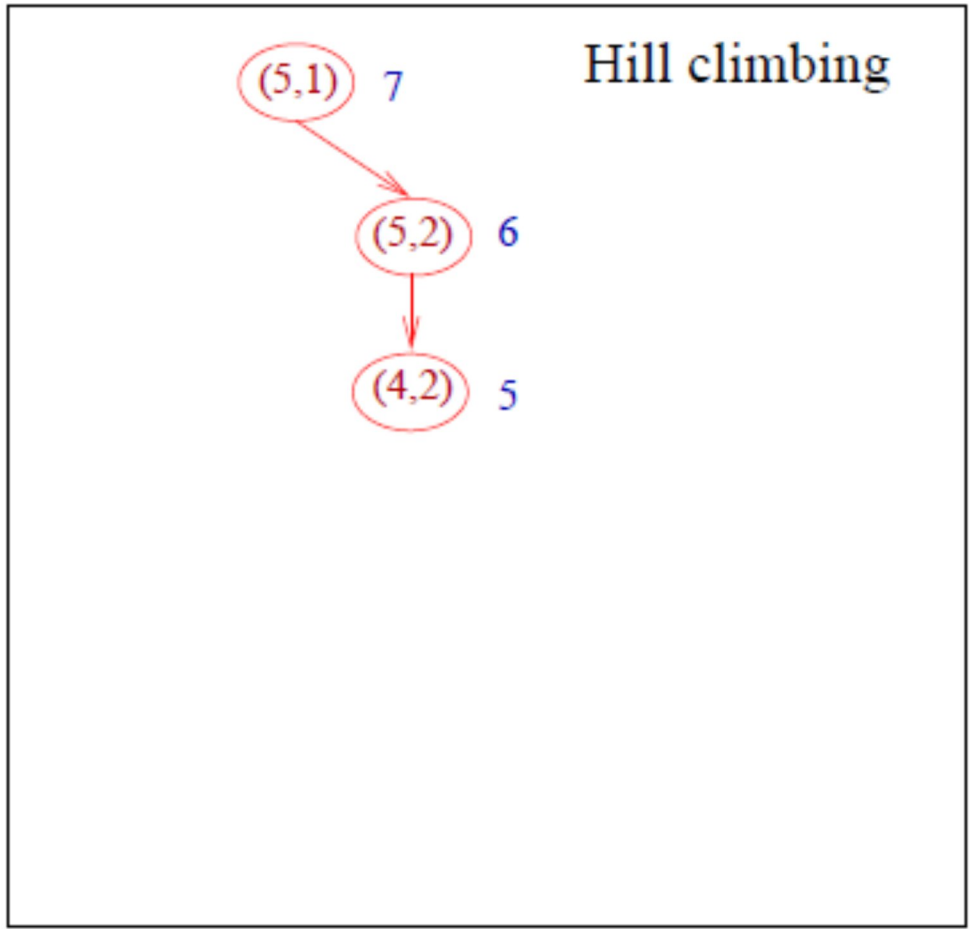




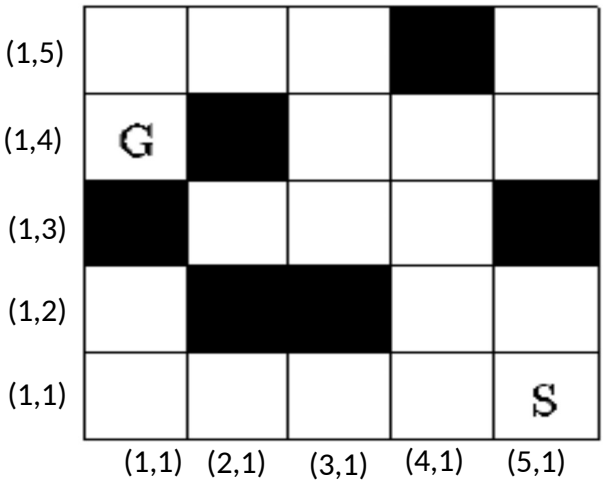
- In the figures describing the steps of the algorithm
- each position will be denoted by a pair of numbers which are the horizontal and vertical coordinates (that is, for (x, y), x denotes the horizontal coordinate and y denotes the vertical coordinate).
- The Manhattan distance between the points (x, y) and (z, t) is given by $d((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2|$.

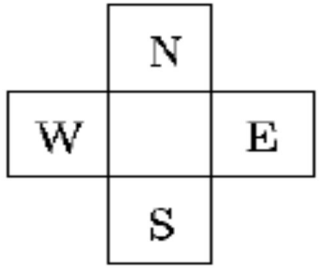


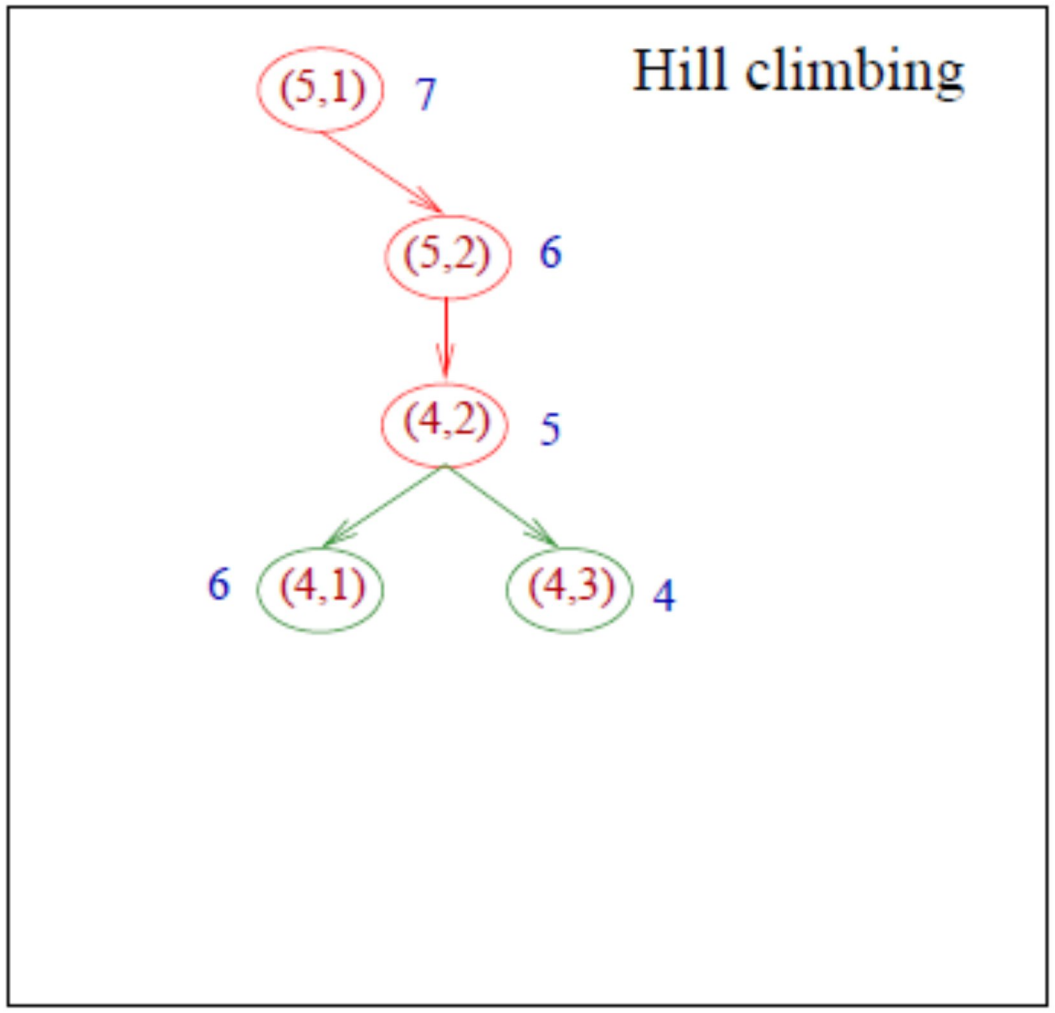




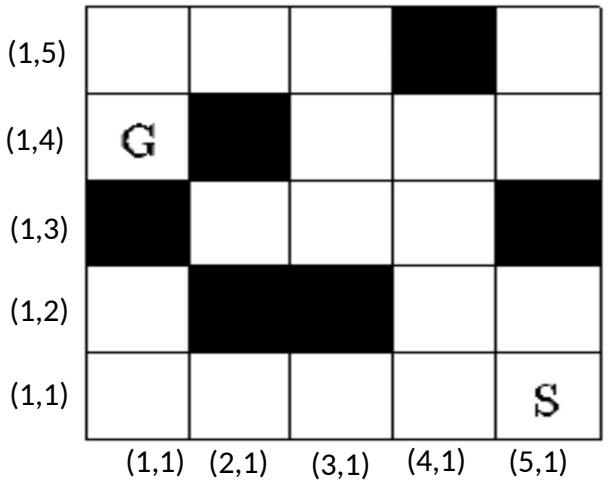
- In the figures describing the steps of the algorithm
- each position will be denoted by a pair of numbers which are the horizontal and vertical coordinates (that is, for (x, y), x denotes the horizontal coordinate and y denotes the vertical coordinate).
- The Manhattan distance between the points (x, y) and (z, t) is given by $d((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2|$.

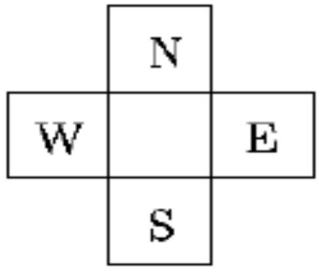


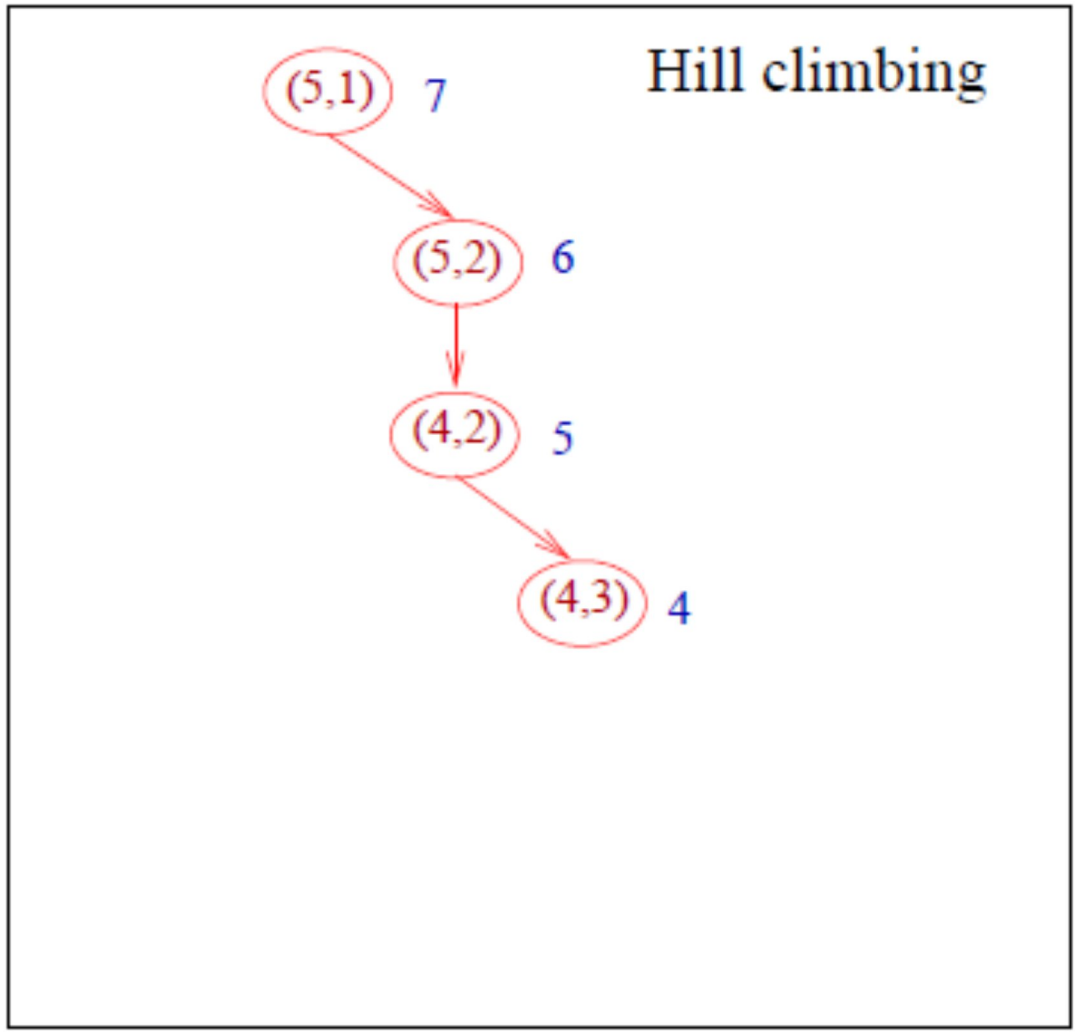




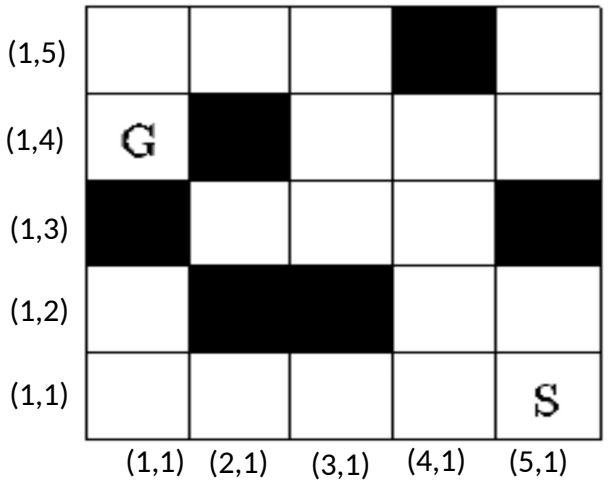
- In the figures describing the steps of the algorithm
- each position will be denoted by a pair of numbers which are the horizontal and vertical coordinates (that is, for (x, y), x denotes the horizontal coordinate and y denotes the vertical coordinate).
- The Manhattan distance between the points (x, y) and (z, t) is given by $d((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2|$.

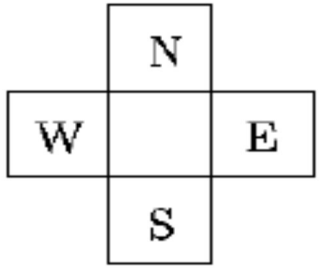


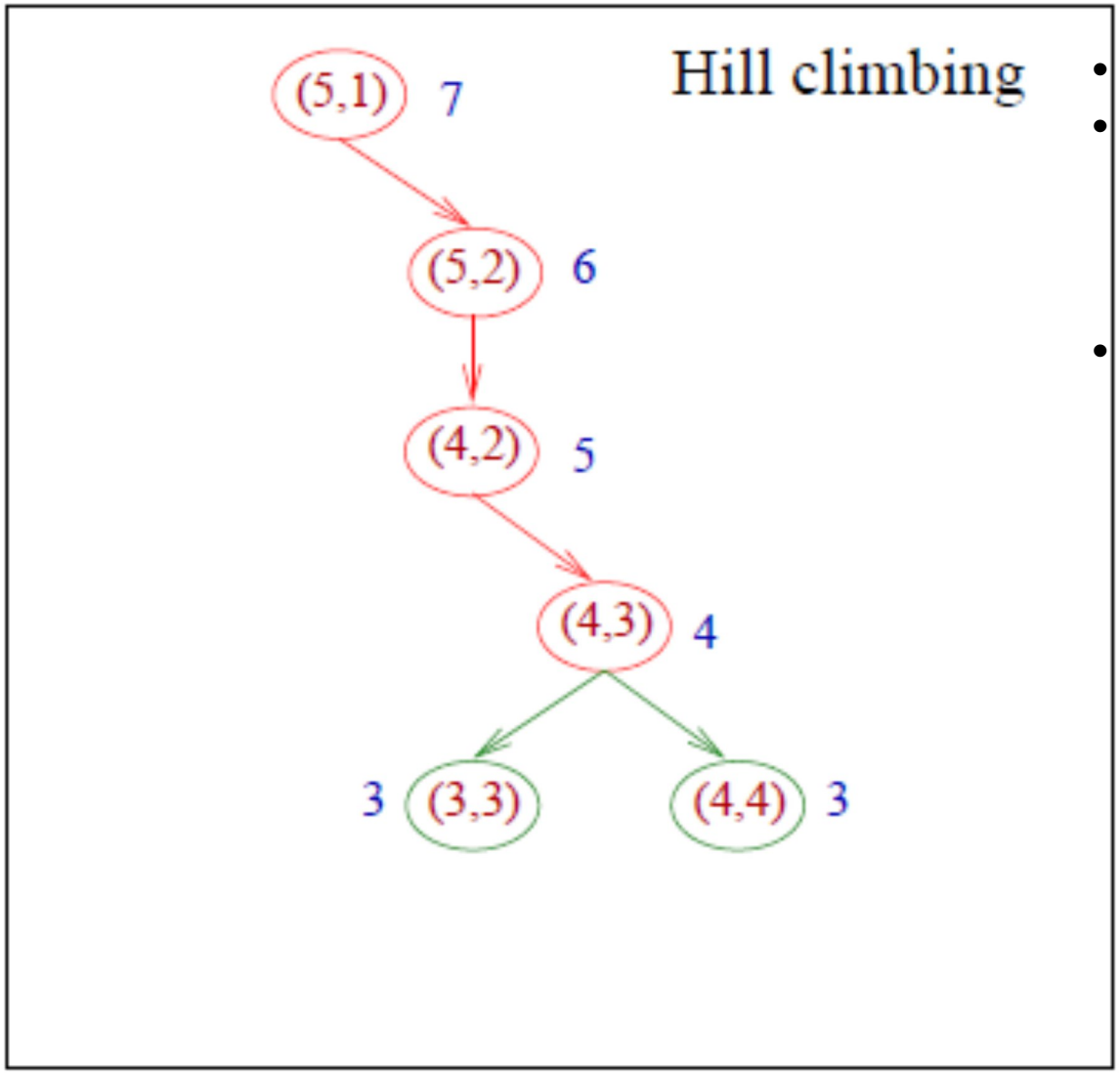




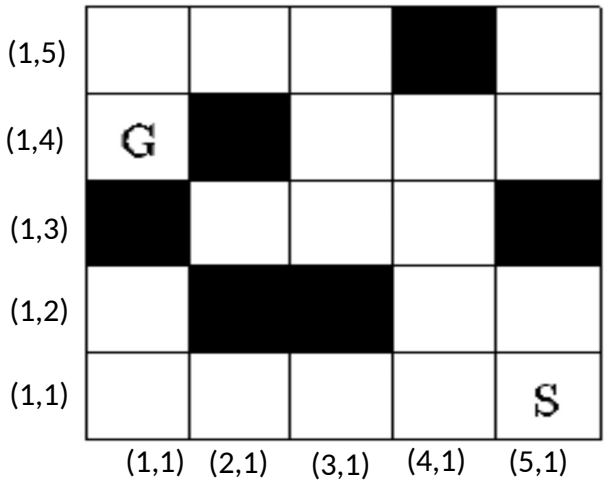
- In the figures describing the steps of the algorithm
- each position will be denoted by a pair of numbers which are the horizontal and vertical coordinates (that is, for (x, y), x denotes the horizontal coordinate and y denotes the vertical coordinate).
- The Manhattan distance between the points (x, y) and (z, t) is given by $d((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2|$.

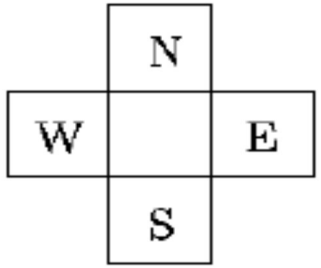


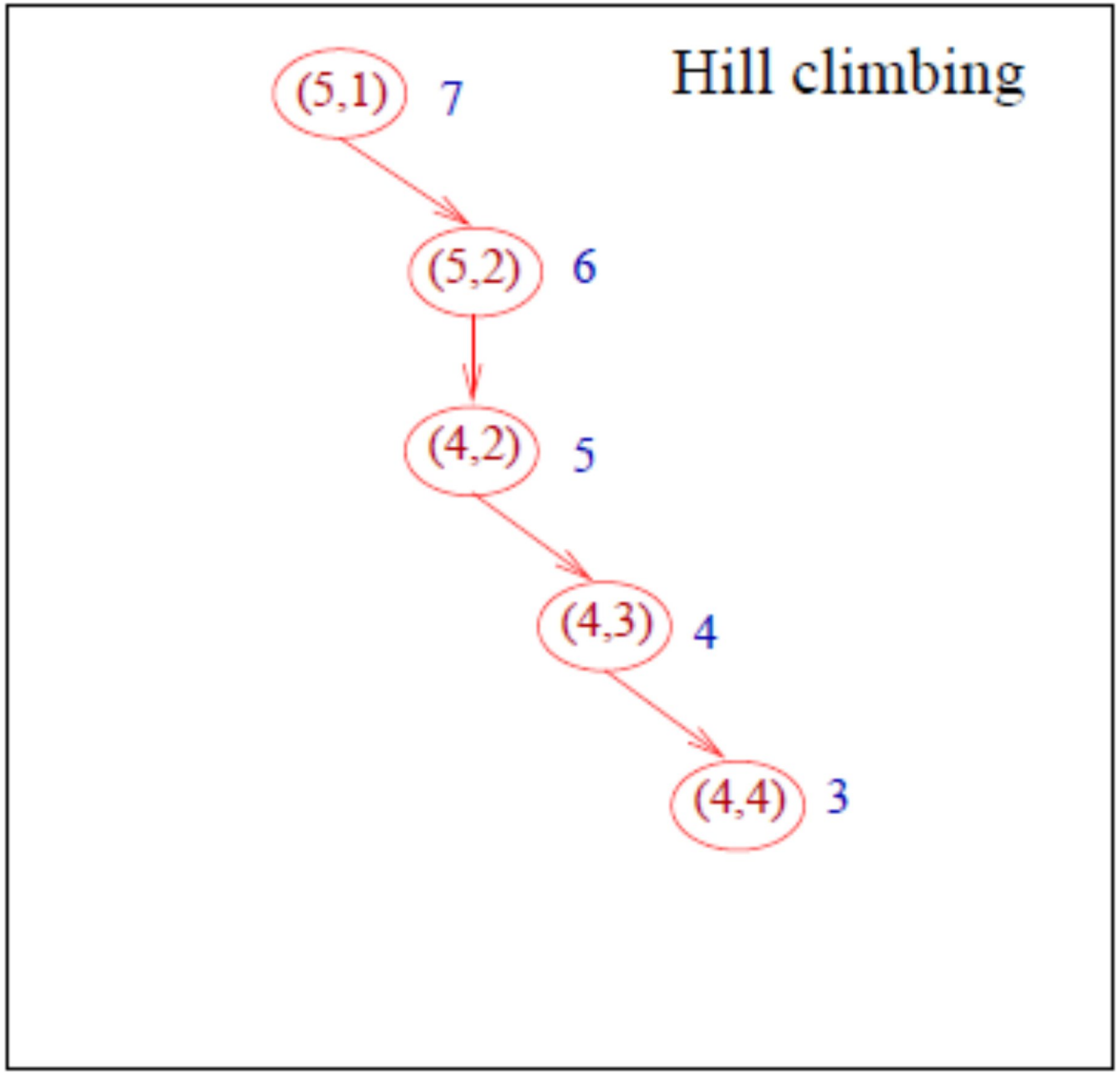




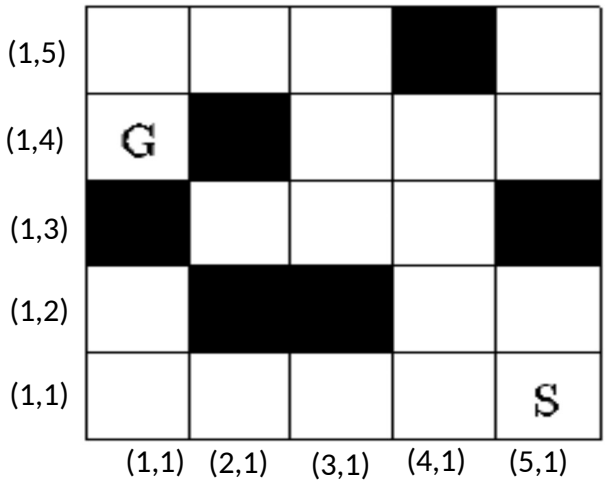
- In the figures describing the steps of the algorithm
- each position will be denoted by a pair of numbers which are the horizontal and vertical coordinates (that is, for (x, y), x denotes the horizontal coordinate and y denotes the vertical coordinate).
- The Manhattan distance between the points (x, y) and (z, t) is given by $d((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2|$.

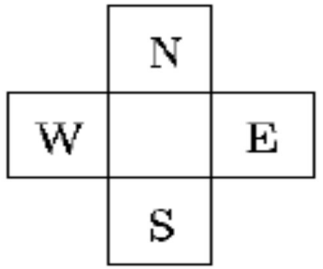


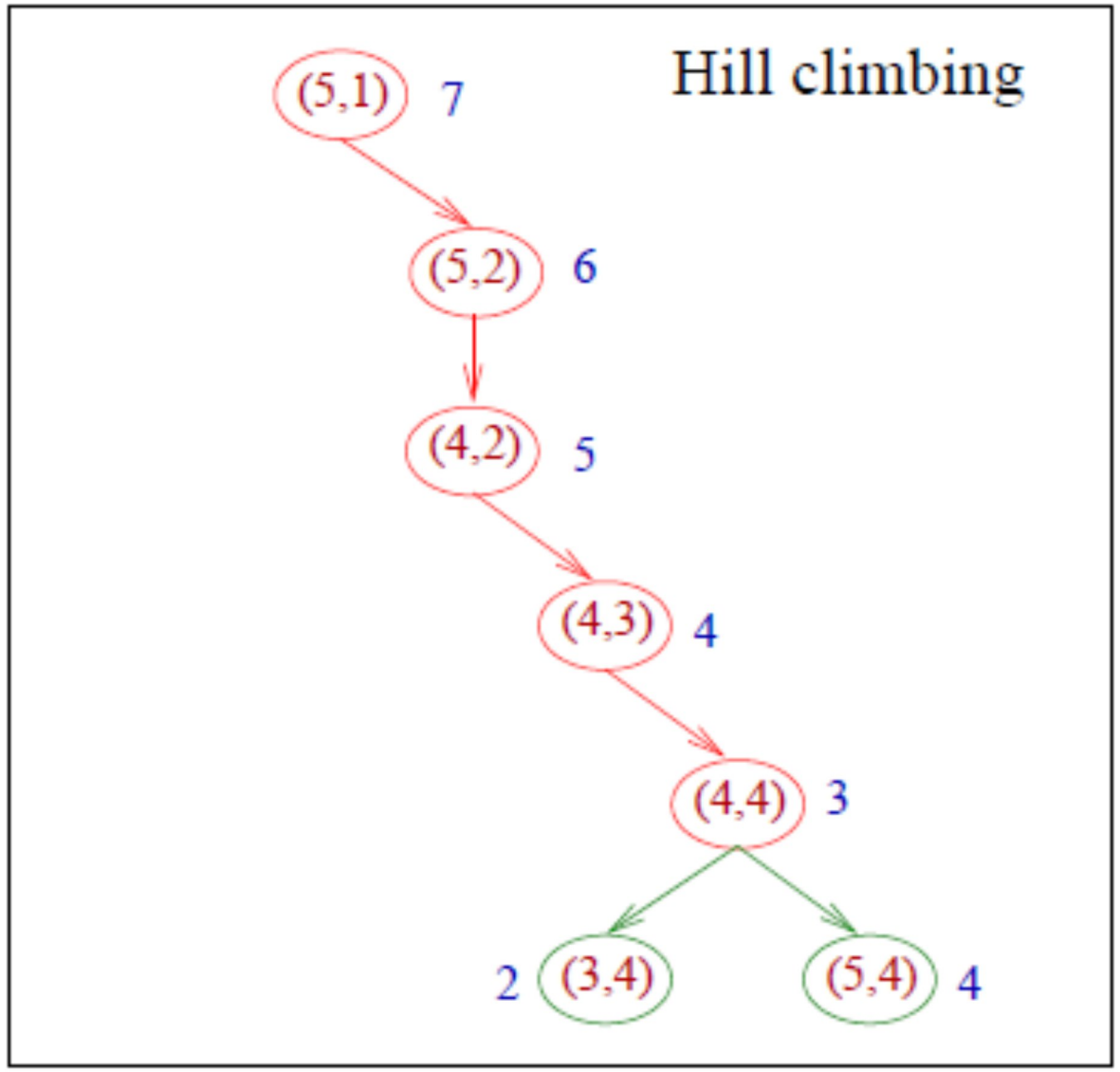




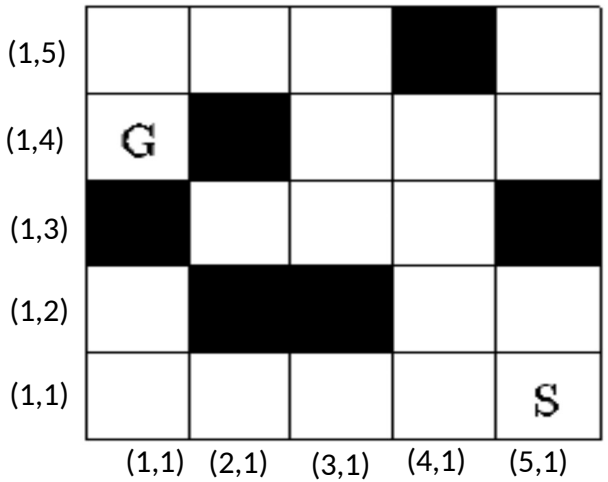
- In the figures describing the steps of the algorithm
- each position will be denoted by a pair of numbers which are the horizontal and vertical coordinates (that is, for (x, y), x denotes the horizontal coordinate and y denotes the vertical coordinate).
- The Manhattan distance between the points (x, y) and (z, t) is given by $d((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2|$.

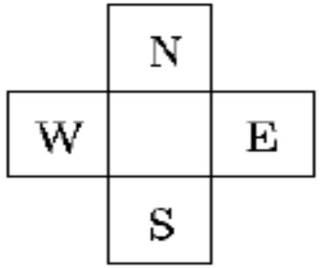






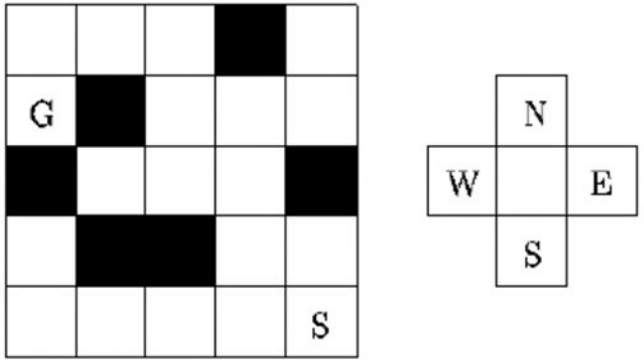
- In the figures describing the steps of the algorithm
- each position will be denoted by a pair of numbers which are the horizontal and vertical coordinates (that is, for (x, y), x denotes the horizontal coordinate and y denotes the vertical coordinate).
- The Manhattan distance between the points (x, y) and (z, t) is given by $d((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2|$.

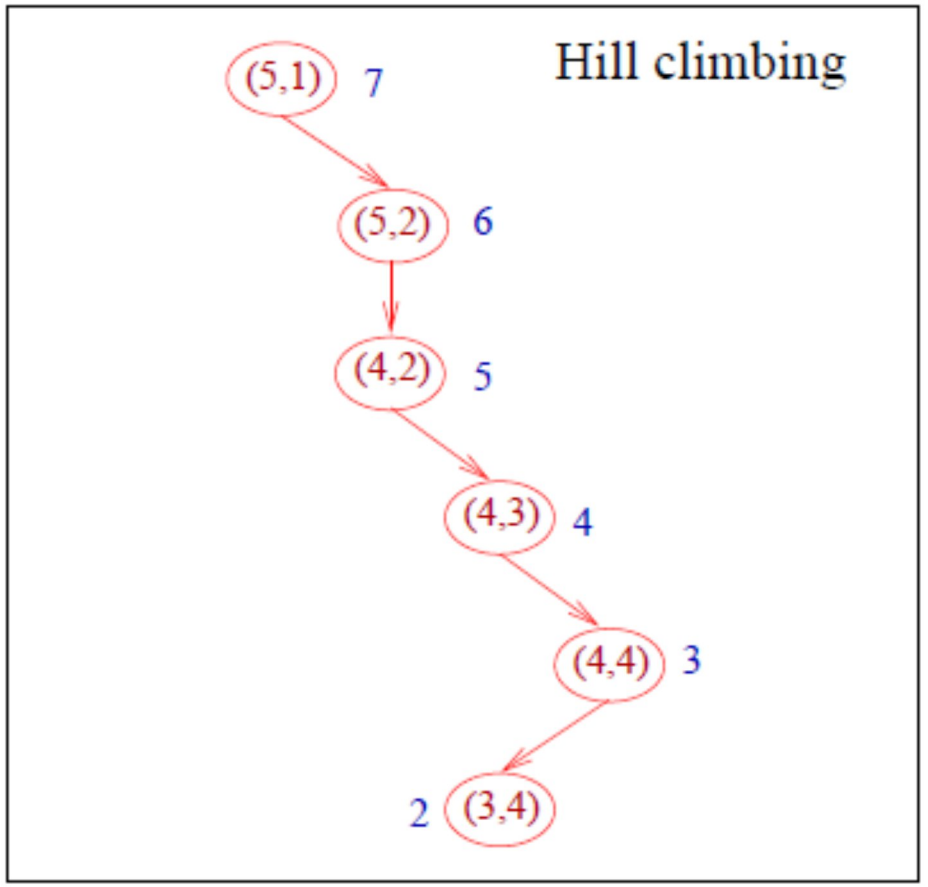




Example of hill-climbing for the maze problem

- The algorithm is stuck in (3,4) because all neigbors of (3,4) are worse than (3,4) (i.e., they have a higher Manhattan distance).





Example of hill-climbing for finding the minimum of a quadratic function

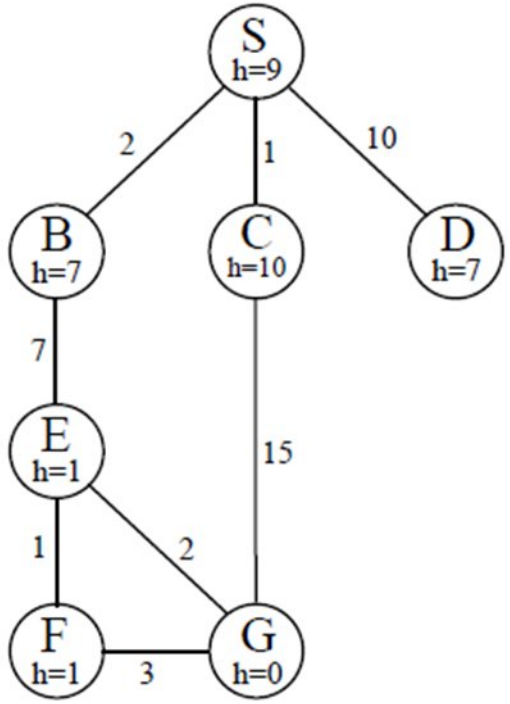
- Consider the function $f(x) = x^2 + 3x + 5$ defined on integer numbers in the interval $[-20,20]$.
- Use the hill-climbing algorithm to find the function's minimum value on this interval.
- First define the search space, then the neighborhood. Try to
  - Find the minimum using starting point $x = 0$
  - Find the minimum using starting point $x = -6$.

Example of hill-climbing for finding the minimum of a quadratic function

- The search space is the set {−20,−19, …,−1, 0, 1, 2, …, 19, 20}.
- The neighborhood for $x = 0$ is $\{-1,0,1\}$, for $x = -1$, $\{-2,-1,0\}$ etc. In general for an x, the neighborhood is $\{x-1,x,x+1\}$.
  - f(0) = 5, f(−1) = 3, f(1) = 9, as we are looking for the minimum, hill climbing will select x = −1
  - f(−1) = 3, f(−2) = 3, f(0) = 5, so the algorithm will stop and give the answer x = −1, with the minimum value f(−1) = 3
- Starting from x = −6,
  - f(−6) = 23, f(−7) = 33, f(−5) = 15, so select x = −5.
  - f(−5) = 15, f(−6) = 23, f(−4) = 9, so x = −4 is selected.
  - f(−4) = 9, f(−5) = 15, f(−3) = 5, so x = −3 is selected.
  - f(−3) = 5, f(−4) = 9, f(−2) = 3, so x = −2 is selected.
  - f(−2) = 3, f(−3) = 5, f(−1) = 3, so x = −2 is returned as solution.

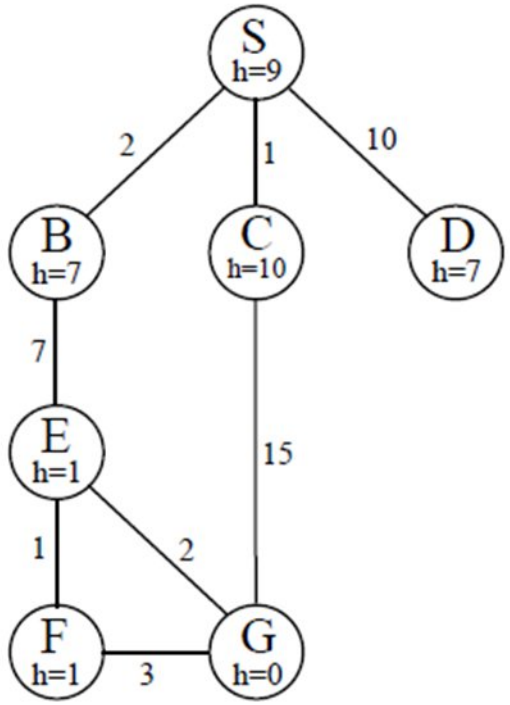
Hill Climbing: Example

	Frontier (Hill Climbing(	Expand	Explored
1			
2			
3			
4			
5			



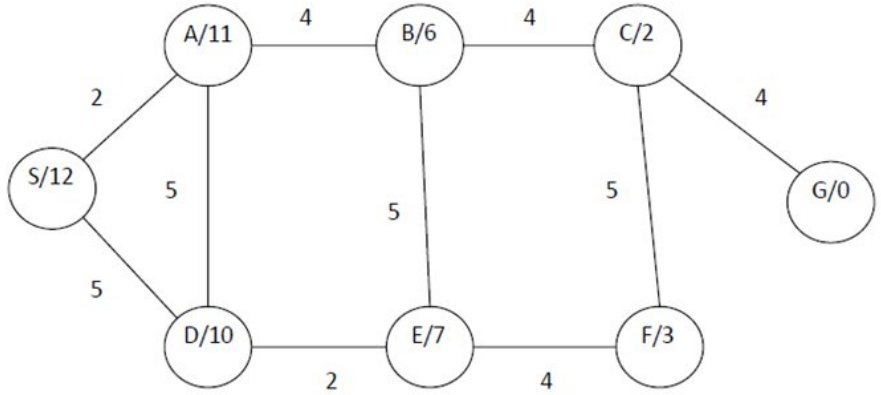
Hill Climbing: Example

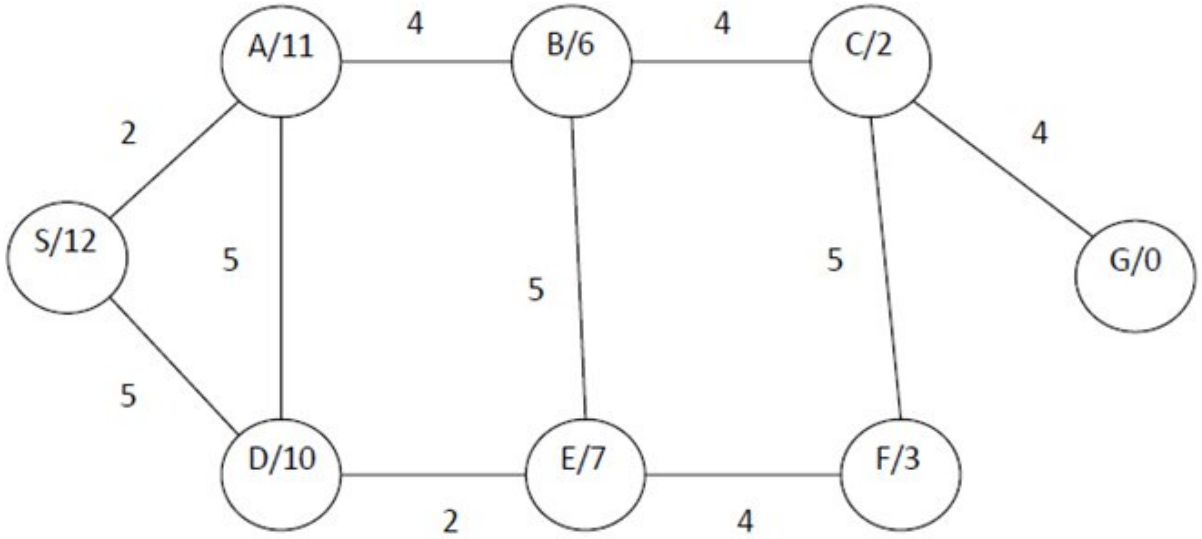
	Frontier (Hill Climbing)	Expand	Explored
1	(S,9)	S	S
2	(B,7)(C,10)(D,7)	B	S,B
3	(E,1)	E	S,B,E
4	(F,1) (G,0)	G	S,B,E,G



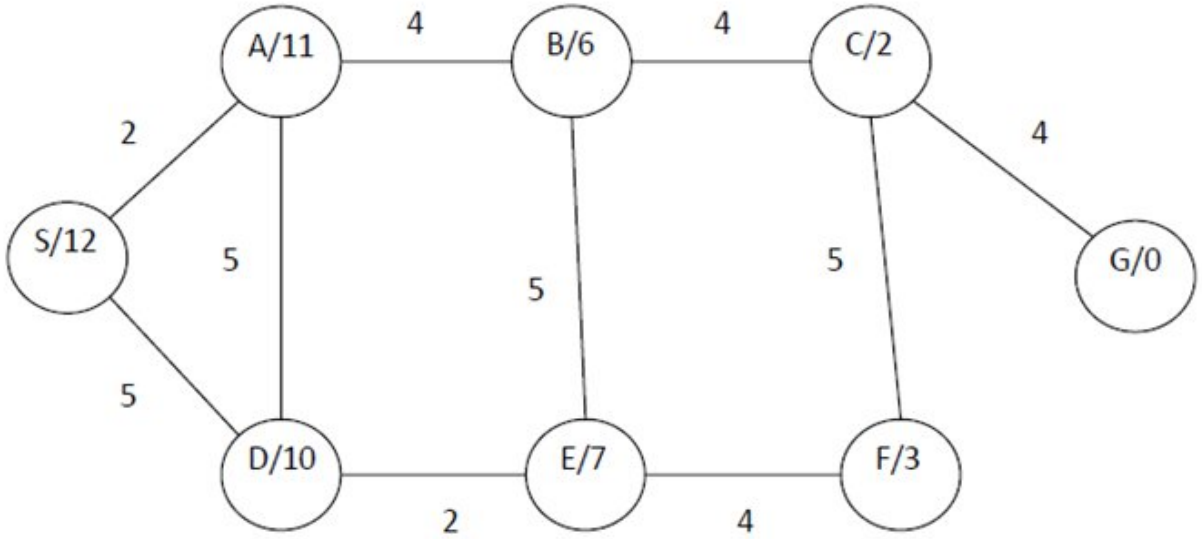
Hill Climbing: Example

	Frontier (Hill Climbing)	Expand
1	(S,12)	S
2		
3		
4		
5		
6		
7		

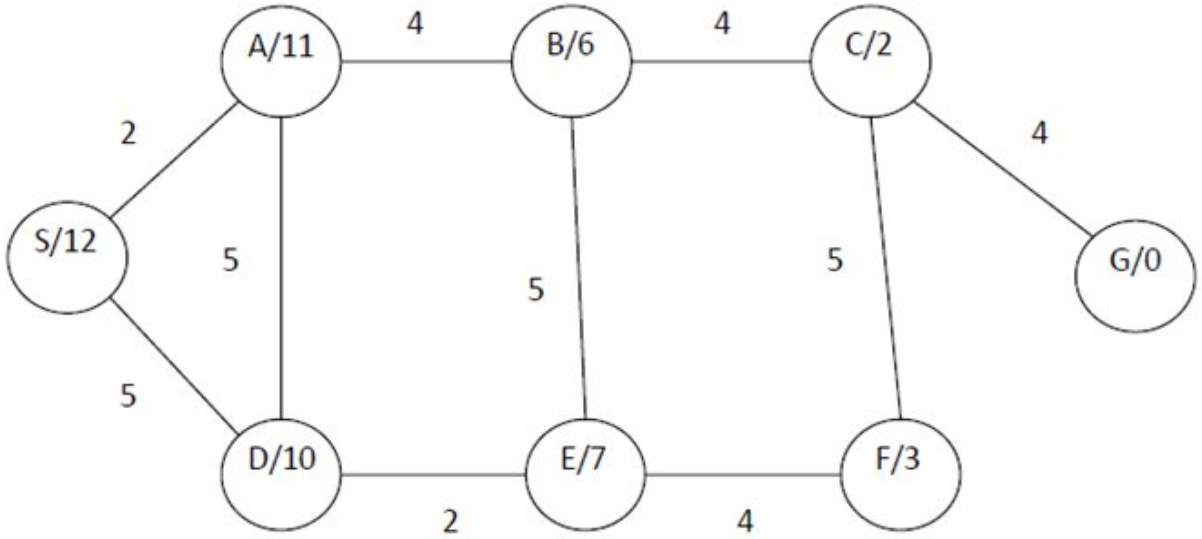




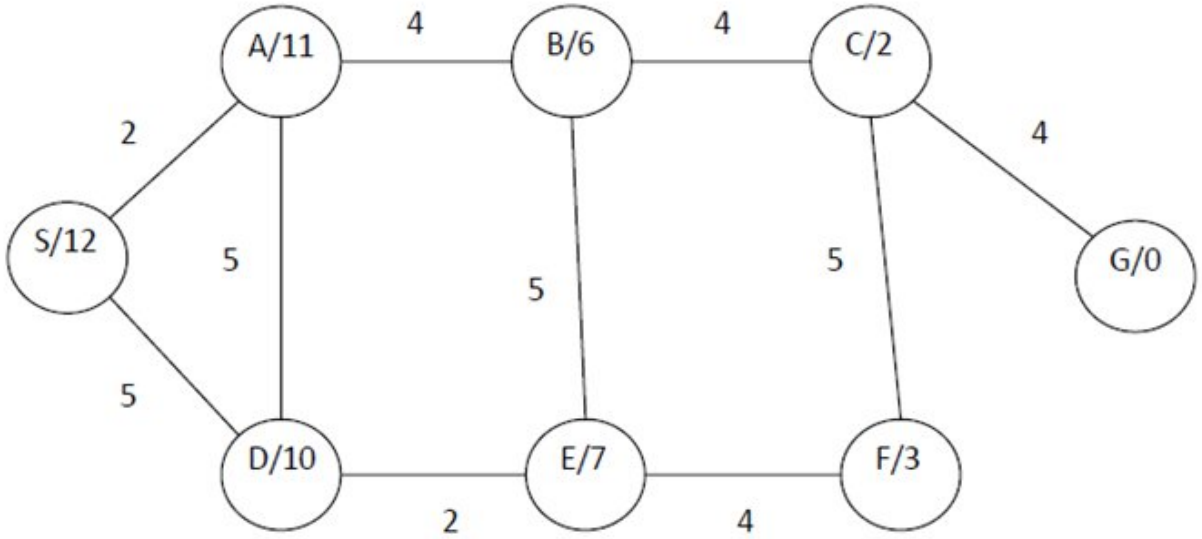
	Frontier (Hill Climbing)	Expand
1	(S,12)	S
2	(A,11)(D,10)	D
3		
4		
5		
6		
7		



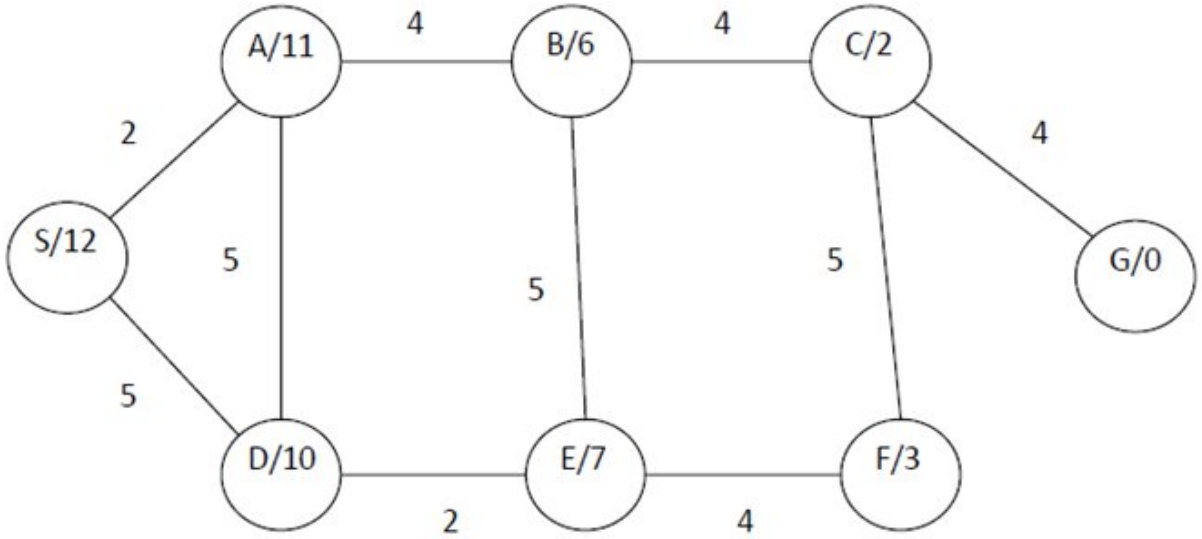
	Frontier (Hill Climbing)	Expand
1	(S,12)	S
2	(A,11)(D,10)	D
3	(S,12) (A,11) (E,7)	E
4		
5		
6		
7		



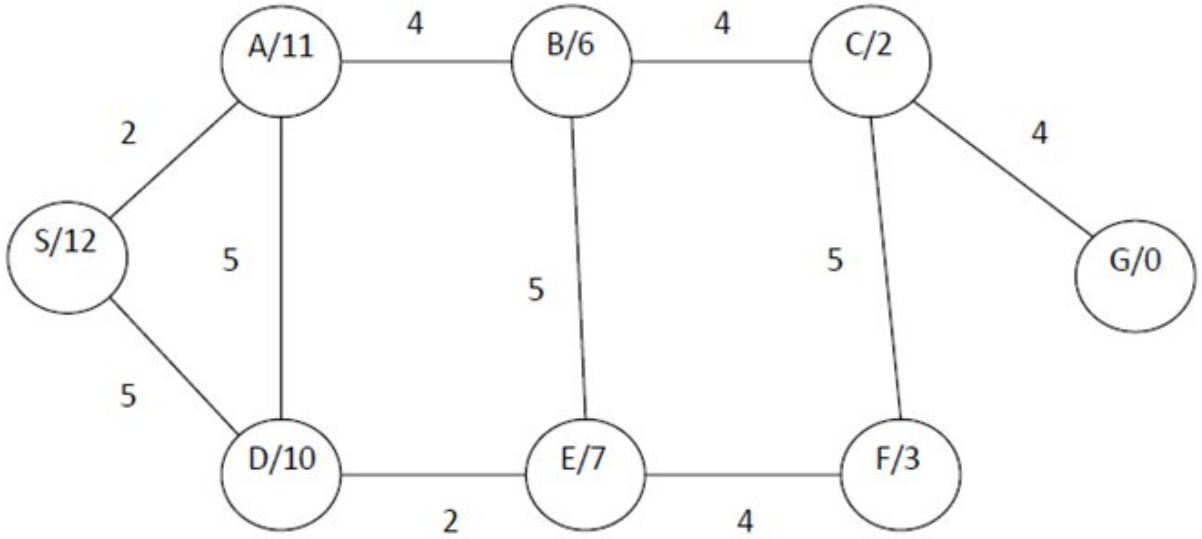
	Frontier (Hill Climbing)	Expand
1	(S,12)	S
2	(A,11)(D,10)	D
3	(S,12) (A,11) (E,7)	E
4	(D,10) (B,6) (F,3)	F
5		
6		
7		



	Frontier (Hill Climbing)	Expand
1	(S,12)	S
2	(A,11)(D,10)	D
3	(S,12) (A,11) (E,7)	E
4	(D,10) (B,6) (F,3)	F
5	(C,2) (E,7)	C
6		
7		



	Frontier (Hill Climbing)	Expand
1	(S,12)	S
2	(A,11)(D,10)	D
3	(S,12) (A,11) (E,7)	E
4	(D,10) (B,6) (F,3)	F
5	(C,2) (E,7)	C
6	(F,3) (B,6) (G,0)	G
7		

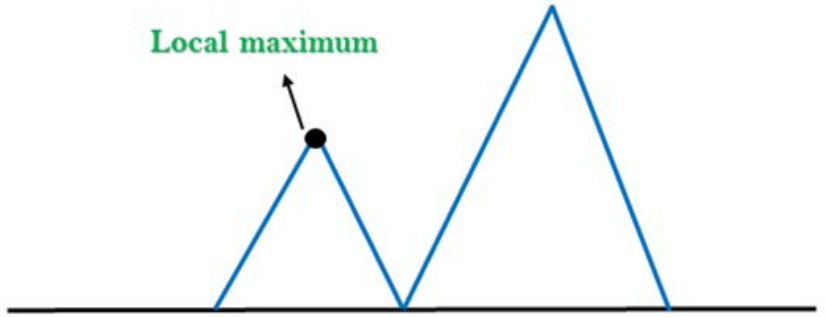


	Frontier (Hill Climbing)	Expand
1	(S,12)	S
2	(A,11)(D,10)	D
3	(S,12) (A,11) (E,7)	E
4	(D,10) (B,6) (F,3)	F
5	(C,2) (E,7)	C
6	(F,3) (B,6) (G,0)	G
7	Goal Node	

Problems with Hill Climbing in AI

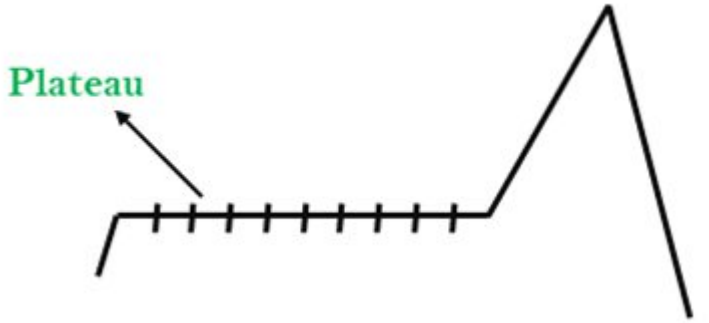
Local Maximum

- A local maximum is a **peak state in the landscape** which is better **than each of its neighboring states**, but there **is another state also present which is higher than the local maximum.**



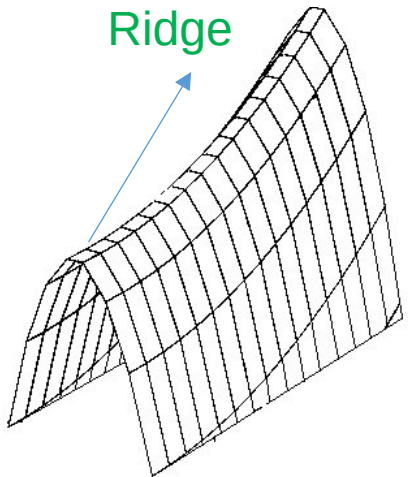
Plateau

- A plateau is the **flat area** of the search space in which all the **neighbor states of the current state contains the same value**, because of this algorithm does not **find any best direction to move**. A hill-climbing search might be **lost** in the plateau area.



Ridge

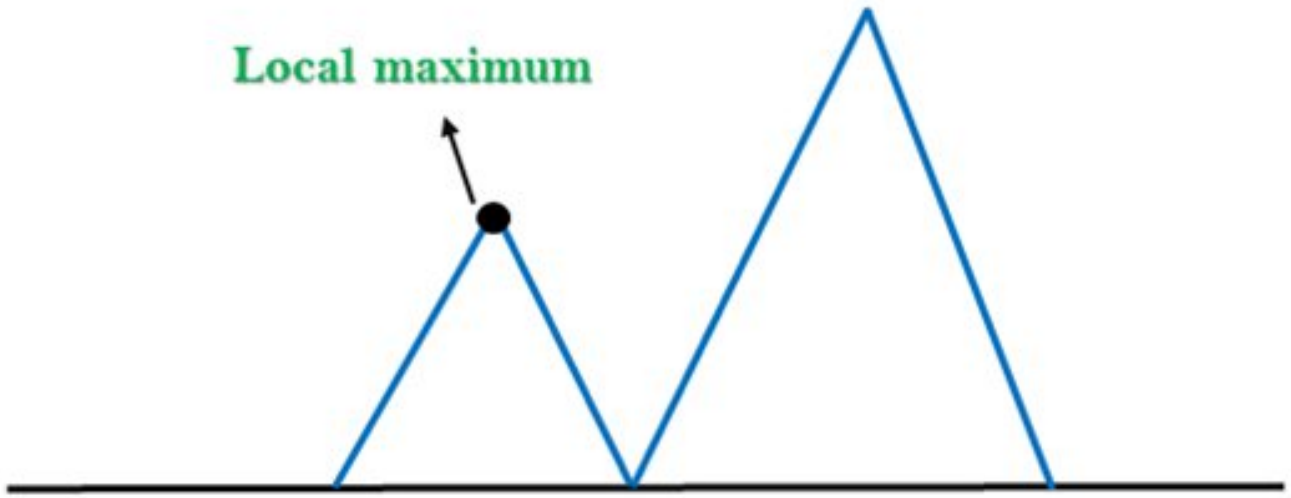
- A ridge is a **special form of the local maximum**. It has an area which is **higher than its surrounding areas**, but itself has a **slope**, and **cannot be reached in a single move.**



Problems with Hill Climbing

1. Local Maximum

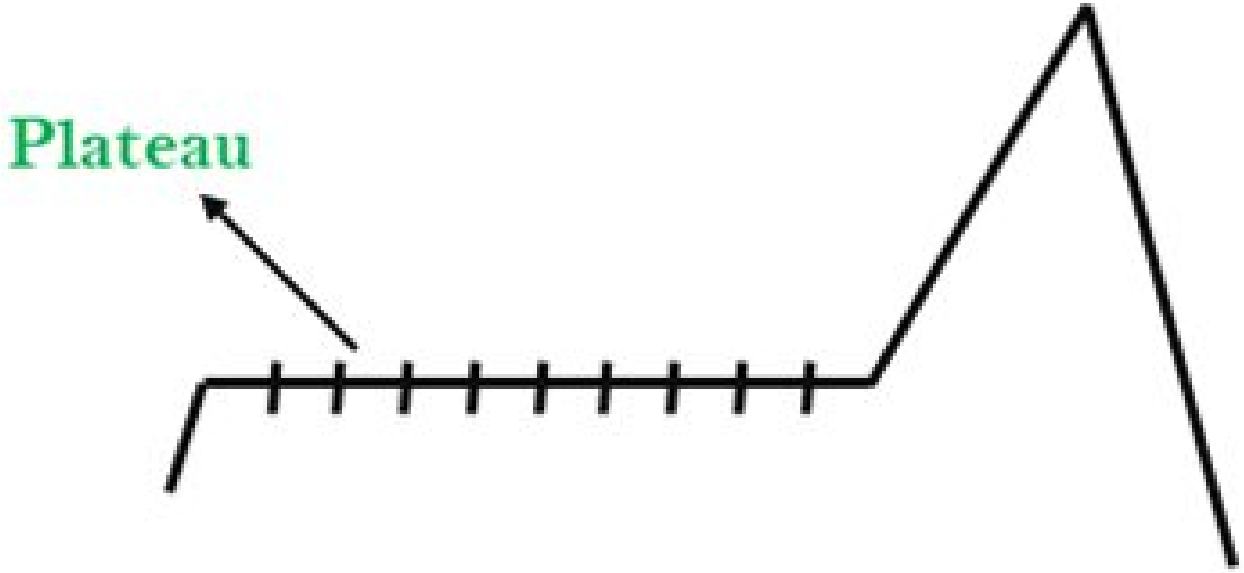
- A **local maximum** is a state that is **better than all its neighboring states**, but **not the best solution overall (global maximum)**



- Hill Climbing stops here because **no neighboring state is better**, even though a better solution exists elsewhere.

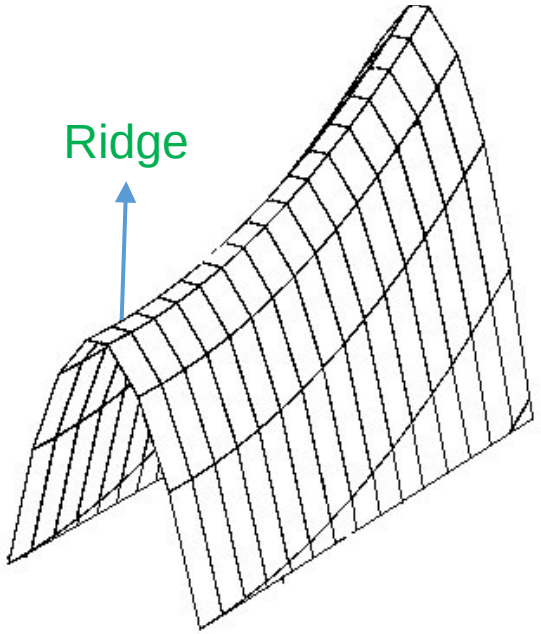
2. Plateau

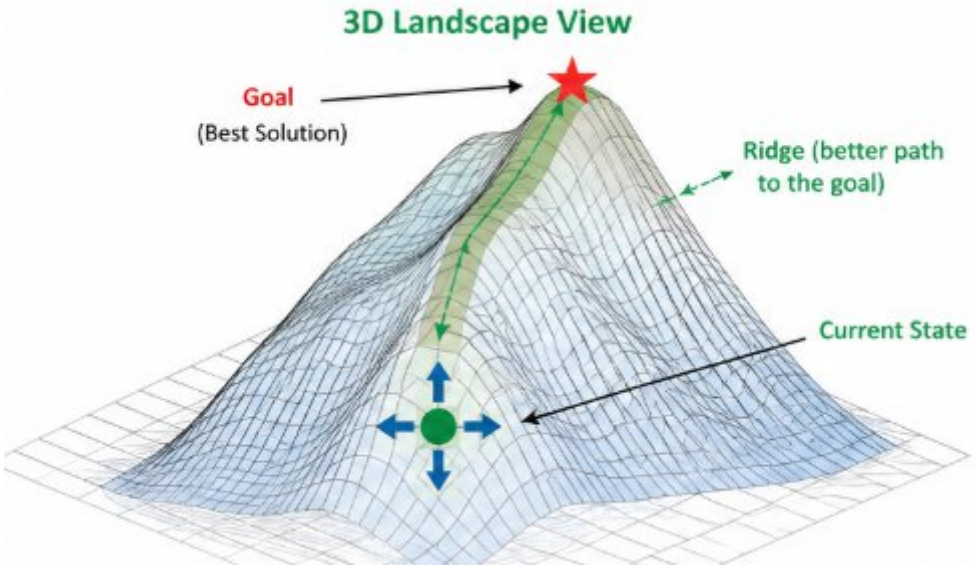
- A **plateau** is a flat region where neighboring states have the **same evaluation value**.



3. Ridge

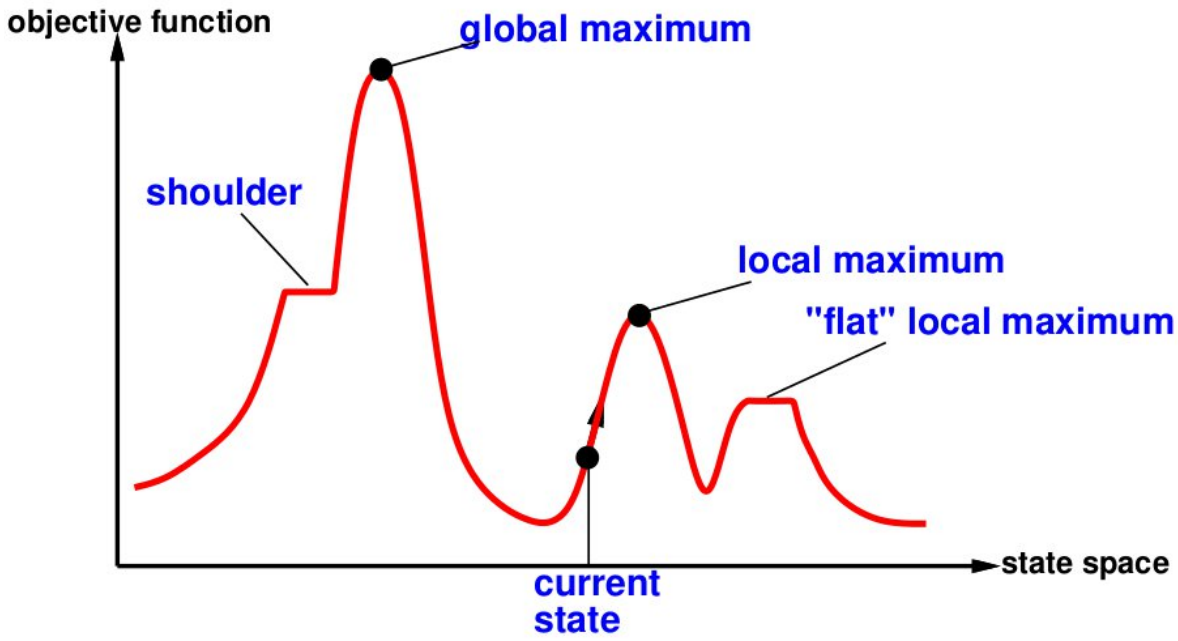
- A **ridge** is a narrow path toward a better solution that cannot be reached with a single local move



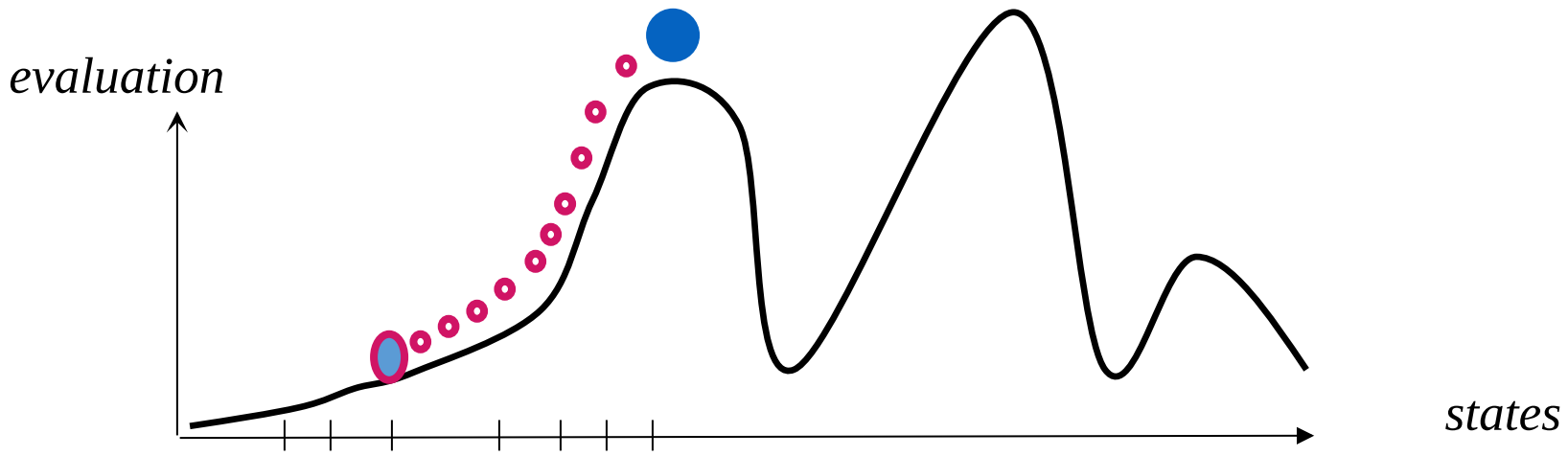


State Space Landscape

- A state space landscape is a graph of states associated with their costs
- A landscape has both “location” (defined by the state) and “elevation” (defined by the value of the heuristic cost function or objective function).



Hill Climbing



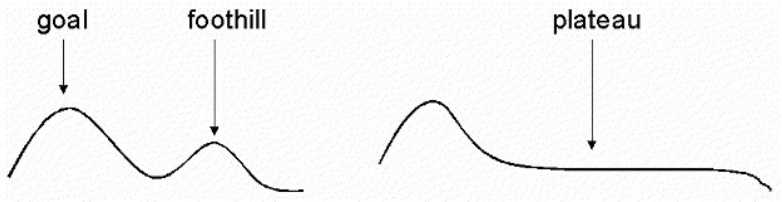
Hill-Climbing – Two Versions

- **Greedy Local Search**
  - Systematically search the Neighborhood.
  - Accept the {first or best} cost decreasing move found.

- **Stochastic Local Search**
  - Choose randomly from amongst the cost-decreasing moves found.

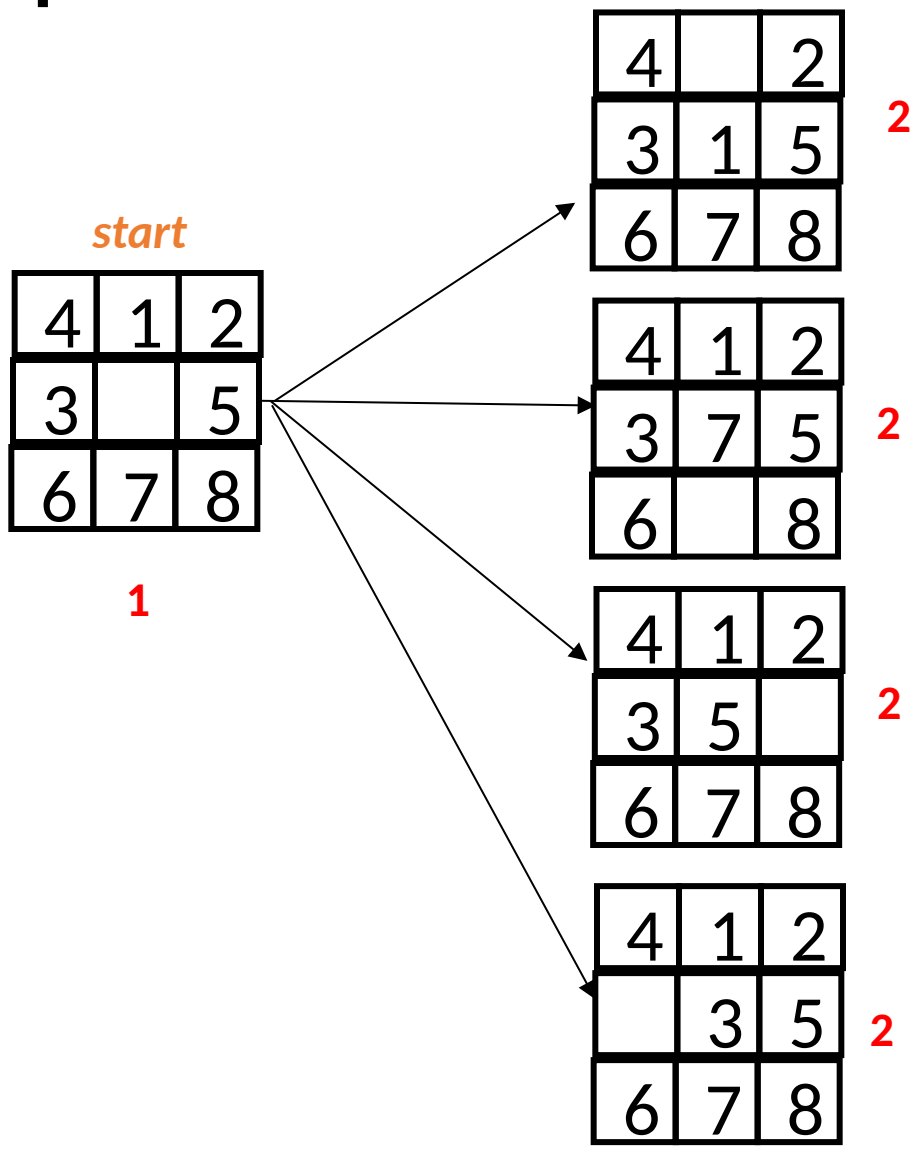
Hill-Climbing Problems

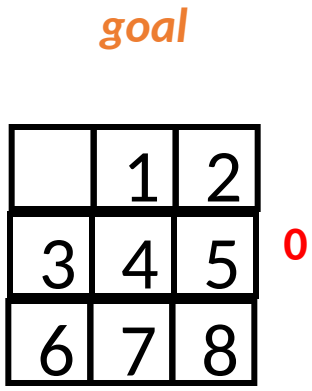
- Greedy Local Search: grabs a good neighbor state without thinking about where to go next
  - However, greedy algorithms do make good progress generally towards the solution



- Unfortunately, hill-climbing
  - Can get stuck in **local maxima**
  - Can be stuck by **ridges** (a series of local maxima that occur close together)
  - Can be stuck by **plateau** (a flat area in the state space landscape)
    - Shoulder: if the flat area rises uphill later on
    - Flat local maximum: no uphill rise exists.

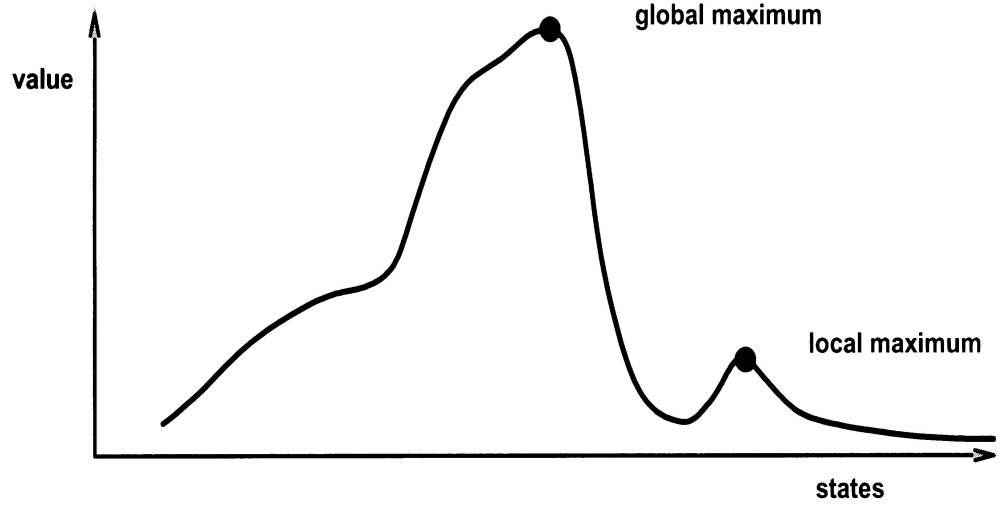
Example of a local "maximum"

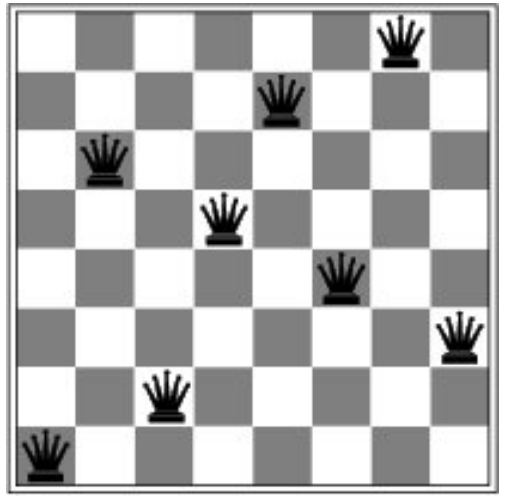




Hill-Climbing Problems

- Local maxima/minima : local search can get stuck on a local maximum/minimum and not find the optimal solution





Local minimum

- Cure
  - + Random restart
  - + Good for Only few local maxima

Improvements

- **Random-restart Hill Climbing**: Selects a series of initial nodes randomly until the solution is found.

- Some problem spaces are great for hill climbing and others are terrible.

Variants of Hill Climbing

- Stochastic hill climbing:
  - chooses at random from among uphill moves
  - converges more slowly, but finds better solutions in some landscapes.
- First-choice hill climbing:
  - generate successors randomly until one is better than the current
  - good when a state has many successors
- Random-restart hill climbing:
  - conducts a series of hill climbing searches from randomly generated initial states, stops when a goal is found
  - It’s complete with probability approaching 1

Variants

- In **simple hill climbing**, the first closer node is chosen, whereas in **steepest ascent hill climbing** all successors are compared and the closest to the solution is chosen. Steepest ascent hill climbing is similar to best-first search, which tries all possible extensions of the current path instead of only one.
- **Stochastic hill climbing** does not examine all neighbors before deciding how to move. Rather, it selects a neighbor at random, and decides (based on the amount of improvement in that neighbor) whether to move to that neighbor or to examine another.
- **Random-restart hill climbing** iteratively does hill-climbing, each time with a random initial condition. The best is kept: if a new run of hill climbing produces a better than the stored state, it replaces the stored state.

Stochastic Hill Climbing

- Stochastic hill climbing chooses at random from among the uphill moves;
- The probability of selection can vary with the steepness of the uphill move.
- Stochastic hill climbing usually converges more slowly than steepest ascent, but in some state landscapes, it finds better solutions.
- Stochastic hill climbing is NOT complete, but it may be less likely to get stuck

First Choice Hill Climbing

- First-choice hill climbing implements stochastic hill climbing by generating successors randomly until one is generated that is better than the current state.
- This is a good strategy when a state has many of successors.
- First-choice hill climbing is also NOT complete
- **Difference from Simple HC:**
  - Doesn’t check systematically
  - Uses random selection

Random Restart Hill Climbing

- Random-Restart Hill Climbing conducts a series of hill-climbing searches from randomly generated initial states, until a goal is found.
- Random-Restart Hill Climbing is complete if infinite (or sufficiently many tries) are allowed.
- If each hill-climbing search has a probability p of success, then the expected number of restarts required is 1/p.
- The success of hill climbing depends very much on the shape of the state-space landscape:
- If there are few local maxima and plateau, random-restart hill climbing will find a good solution very quickly.
- On the other hand, many real problems have many local maxima to get stuck on.

Nature-Inspired Heuristic Algorithms

- **Genetic Algorithm** (evolution)
- Neural Network (artificial neural network)
- Simulated Annealing (metal annealing)
- Tabu Search (animal's brain)
- Ant Colony Optimization
- Bee Colony Optimization